

蓝牙小马-AI 辅助零编程教程

目录：

project0：e² studio 获取与安装

- 1.对话 AI 导师
- 2.e² studio 软件介绍
- 3.获取安装包
- 4.软件安装

project1：开发板测试和 Programmer 烧录程序

- 1.对话 AI 导师
- 2.概述
- 3.硬件准备
- 4.获取事例程序
- 5.Programmer 理解与应用
- 6.烧录步骤

Project2：AI 导师的获取与应用

- 1.获取 AI 导师
- 2.了解 AI 导师

Project3：AI 导师辅助项目创建

- 1.项目介绍
- 2.创建工程

Project4：AI 导师辅助时钟树配置

- 1.了解时钟
- 2.对话 AI 导师
- 3.时钟树配置完成页面

Project5：AI 导师辅助调试引脚配置

- 1.对话 AI 导师

2.调试引脚配置完成页面

3.原理解释

Project6: AI 辅助定时器配置

1.了解定时器

2.对话 AI 导师

3.创建组件

4.属性配置

5.属性介绍

Project7: AI 辅助串口配置

1.了解串口

2.对话 AI 导师

3.创建组件

4.属性配置

5.属性介绍

Project8: AI 辅助 IC2 配置

1.了解 IC2

2.对话 AI 导师

3.创建组件

4.属性配置

5.属性介绍

Project9: AI 辅助编写 PWM 驱动

1.了解 PWM 相关函数

2.对话 AI 导师

3.src 文件位置

4.创建源文件与头文件

5.代码移植

Project10: AI 辅助编写动作逻辑

1.了解舵机的工作机制

2.对话 AI 导师

Project11:AI 辅助驱动 OLED

- 1.获取驱动
- 2.驱动文件简介
- 3.IC2 相关函数介绍
- 4.OLED 驱动相关函数介绍

Project12:文字与图像取模

- 1.概述
- 2.对话 AI 导师
- 3.字符取模
- 4.对话 AI 导师
- 5.图像转 BMP 格式
- 6.图片取模

Project13:AI 辅助表情显示编写

- 1.对话 AI 导师
- 2.创建源文件与头文件
- 3.代码移植

Project14：主函数编写

- 1.对话 AI 导师
- 2.程序编译
- 3.程序烧录

Project15：蓝牙串口小程序的使用

project0：e² studio 获取与安装

1.对话 AI 导师

⚠ 系统就绪。请在输入框输入 '教我做机器马' 开始。

[21:29] 我: 教我做机器马

[21:29] 导师: 好的! 很高兴能带你一起完成这个酷炫的蓝牙电子小马项目! 我们将一步一步来, 确保你完全掌握每一个环节。

****第一步: 开发板测试与 Programmer 烧录教学****

在正式开始编写代码之前, 我们先来确保你的开发板和烧录工具都准备就绪。

1. ****主控芯片确认****: 本项目选用的是 ****R7FA4M2AD3CFL**** 这款 MCU。
2. ****获取示例程序****: 请先访问 Gitee 仓库: [https://gitee.com/CoreMaker/RA4M2_MINI](https://gitee.com/CoreMaker/RA4M2_MINI), 点击克隆/下载, 选择 "Download ZIP" 将代码下载到你的电脑并解压。
3. ****下载 Renesas Flash Programmer (RFP)****: 请访问 <https://www.renesas.cn/zh/software-tool/renesas-flash-programmer-programming-gui> 下载并安装 RFP 软件。

2.e² studio 软件介绍

瑞萨电子灵活配置软件包 (FSP) 是用于嵌入式系统设计的高质量增强型软件包, 支持瑞萨电子 RA 产品家族 Arm 微控制器, 提供用户友好的界面且可灵活扩展, 确保从入门级到高性能的整个 RA 微控制器的软件兼容性。在全新 Arm® TrustZone® 和其他高级安全功能的支持下, FSP 提供一种快速、通用的方式, 使用量产级驱动程序、Azure RTOS、FreeRTOS™ 和其他中间件协议栈来构建安全、互联的物联网设备。

3.获取安装包

下载地址: <https://www.renesas.cn/zh/software-tool/flexible-software-package-fsp>

点击连接后跳转入如下页面:

概览

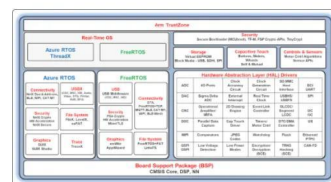
描述特性发布信息目标设备

瑞萨电子灵活配置软件包 (FSP) 是用于嵌入式系统设计的高质量增强型软件包，支持瑞萨电子 RA 产品家族 Arm 微控制器，提供用户友好的界面且可灵活扩展，确保从入门级到高性能的整个 RA 微控制器的软件兼容性。在全新 Arm® TrustZone® 和其他高级安全功能的支持下，FSP 提供一种快速、通用的方式，使用量产级驱动程序、Azure RTOS、FreeRTOS™ 和其他中间件协议栈来构建安全、互联的物联网设备。

下载最新FSP (v5.9.0):

FSP 平台安装程序 (包含 e² studio 集成开发环境、工具链和 FSP 软件包) :

- FSP Windows 平台安装程序
- FSP Linux 平台安装程序
- FSP macOS 平台安装程序



点击 [FSP Windows 平台安装程序](#)，在信息收集界面输入个人信息即可下载：

RENESAS

灵活配置软件包 (FSP) 下载

填写表格以完成您的灵活配置软件包 (FSP) 下载

瑞萨电子RA灵活配置软件包 (FSP) 提供了一种使用瑞萨RA系列Arm微控制器 (MCU) 构建安全连接的物联网 (IoT) 设备的快速通用方法。FSP提供生产就绪的外围设备驱动程序、Azure RTOS、FreeRTOS和中间件协议栈，以利用FSP生态系统。



电子邮件

[这不是我](#)

国家/地区

姓

名

公司

地址

城市

省/自治区

邮编

联系电话

Operating System

Windows

Installer

☐ e² studio

实时操作系统 (RTOS) 选择

☐ Azure RTOS ☐ FreeRTOS ☒ 不使用实时操作系统

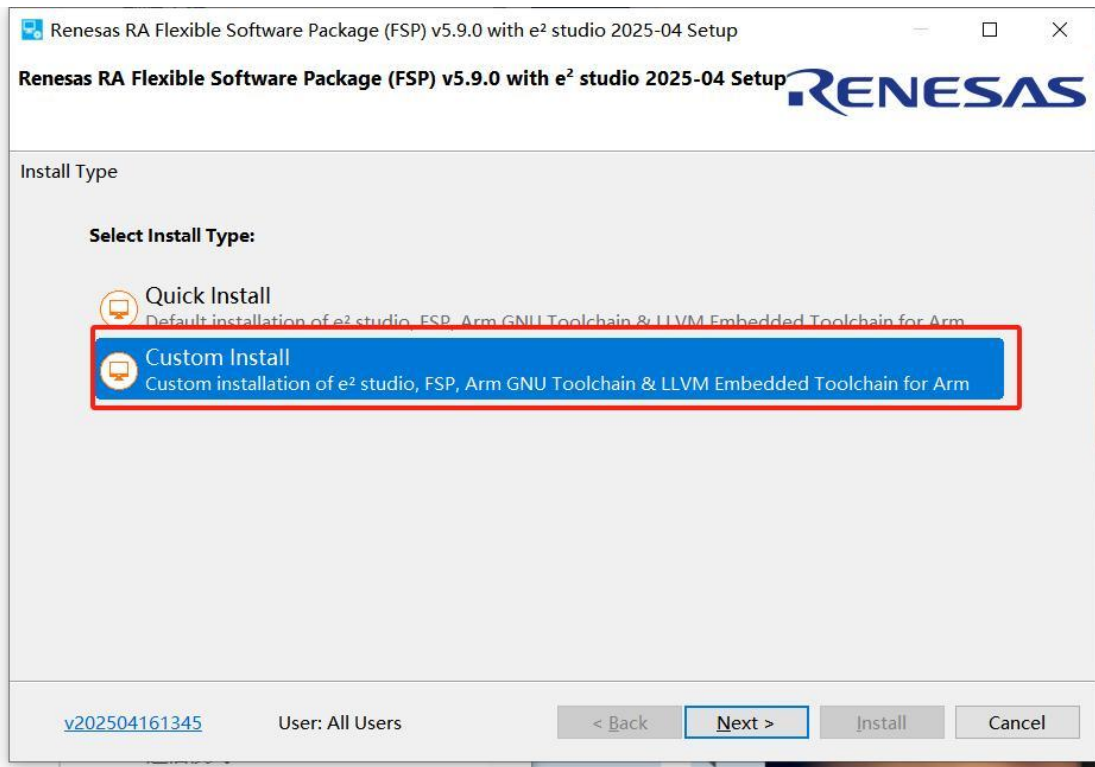
☒ 我同意接受瑞萨电子网站政策条款和隐私条款。

完整安装包应当是 1.6G 以上：

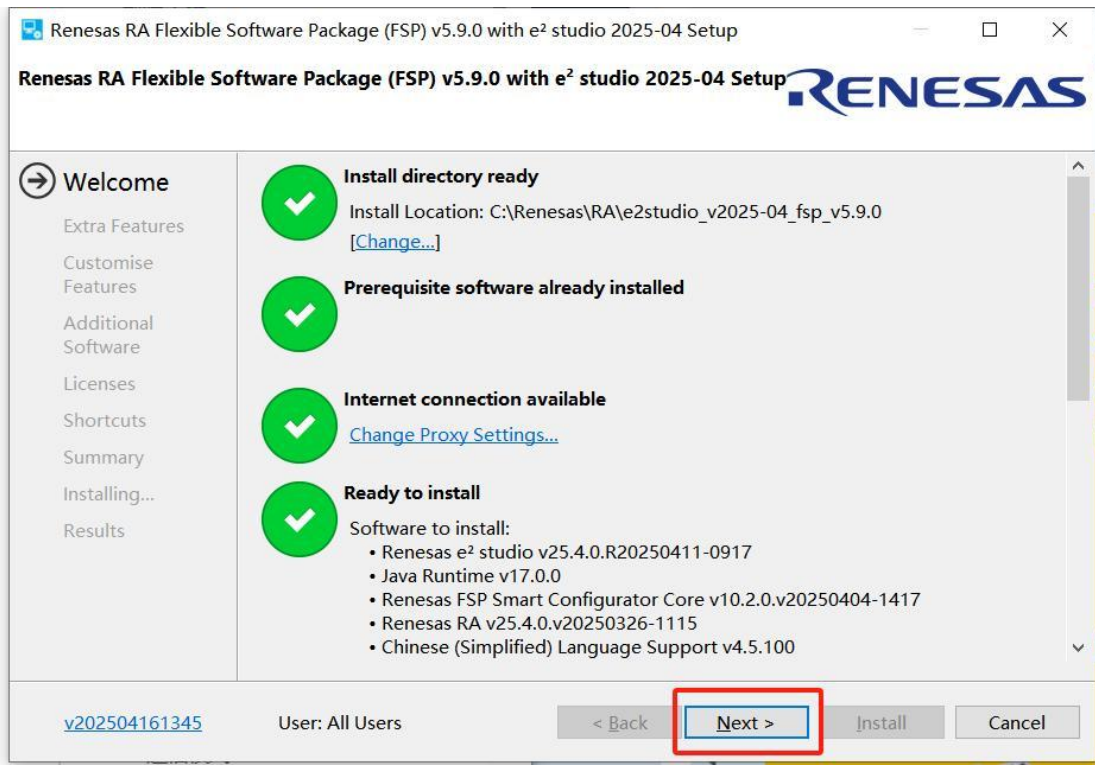


4. 软件安装

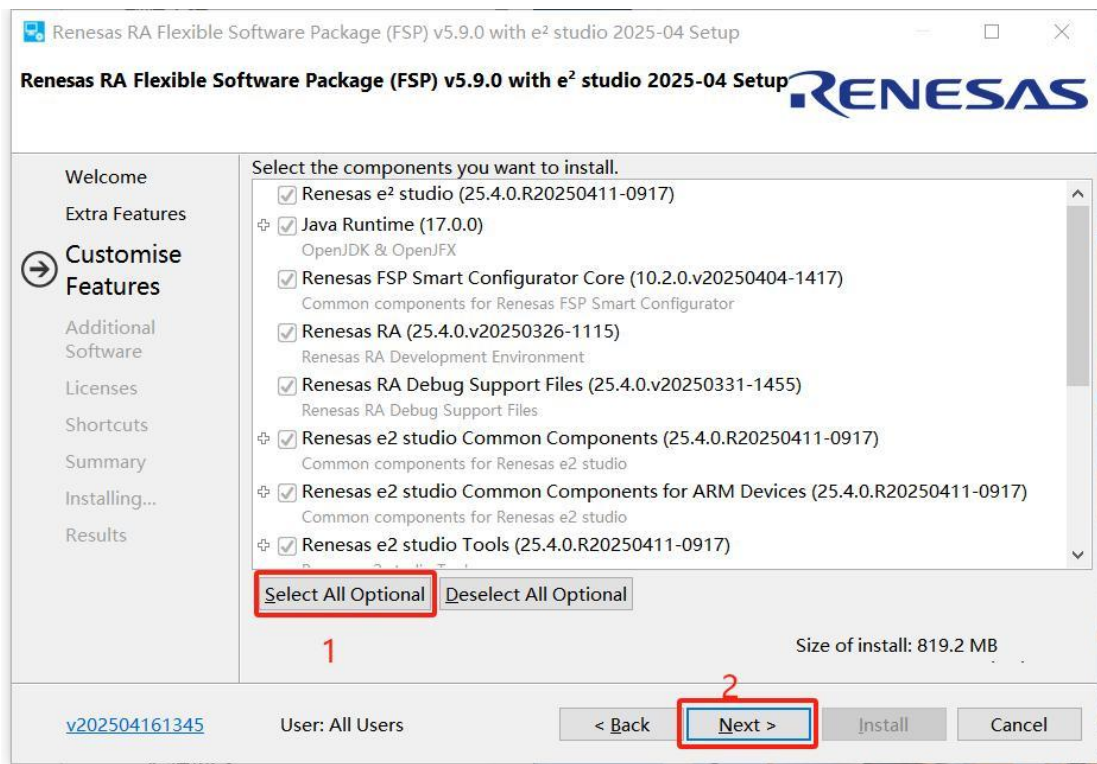
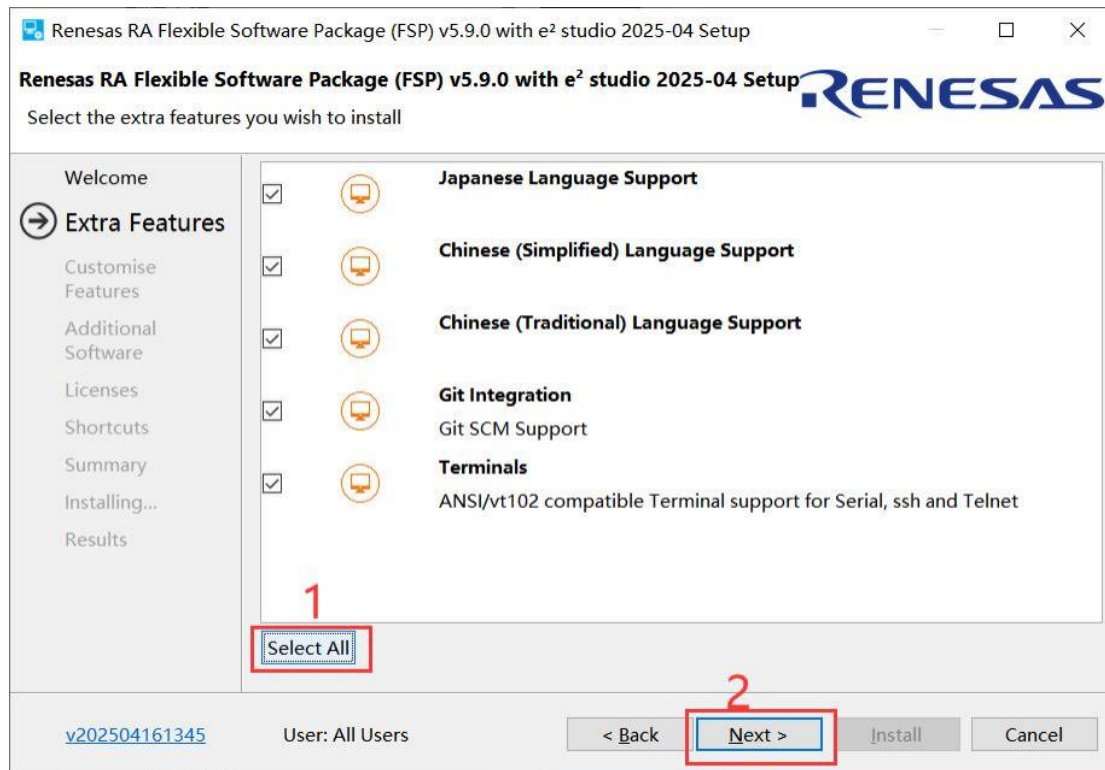
下载完成后点击安装包开始安装，这里首先选择自定义安装：

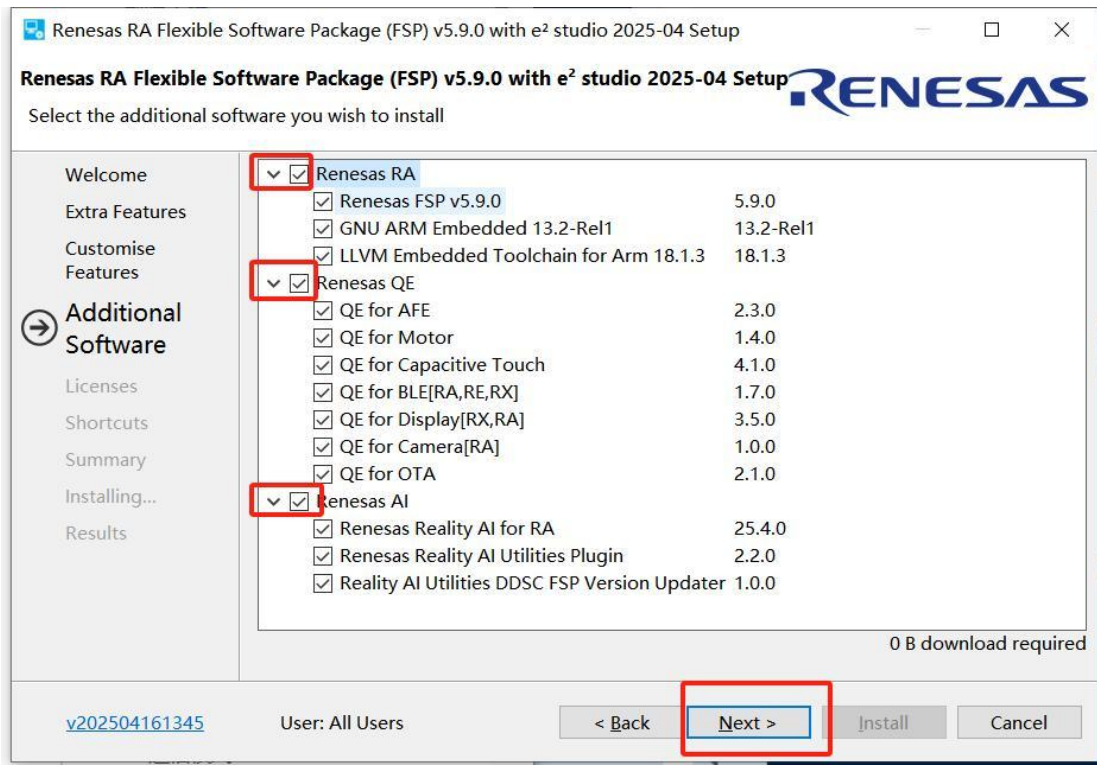


检测安装和配置开发环境，没问题点击下一步：

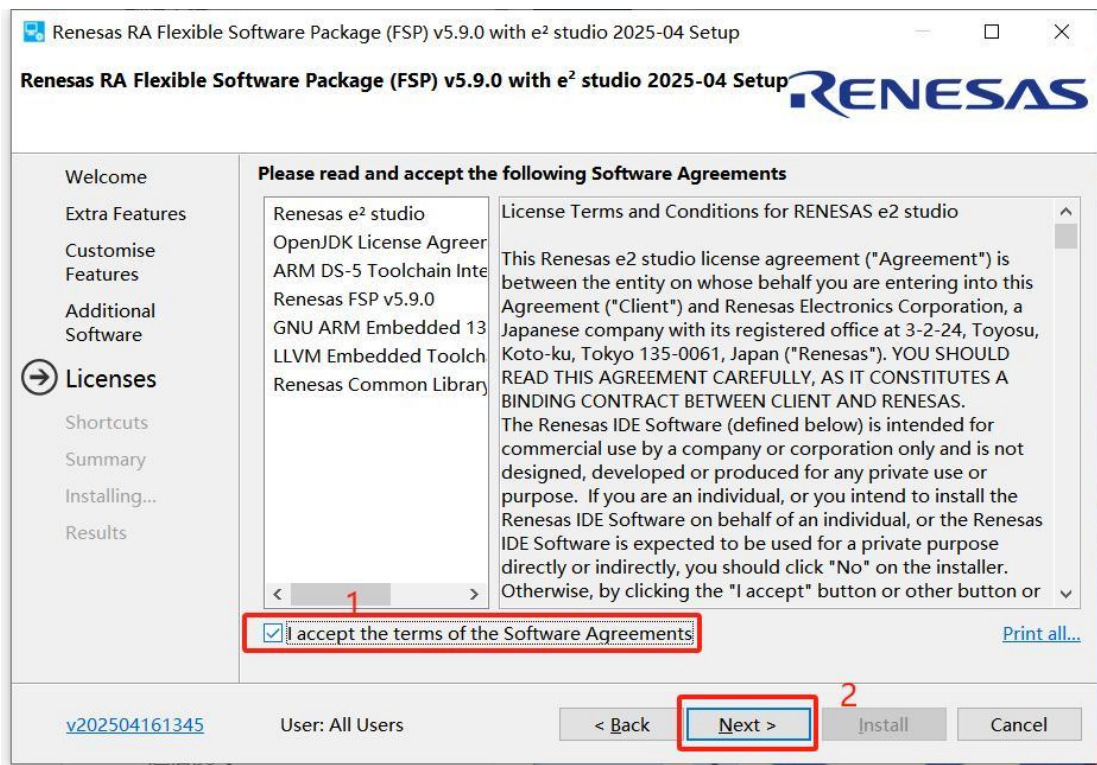


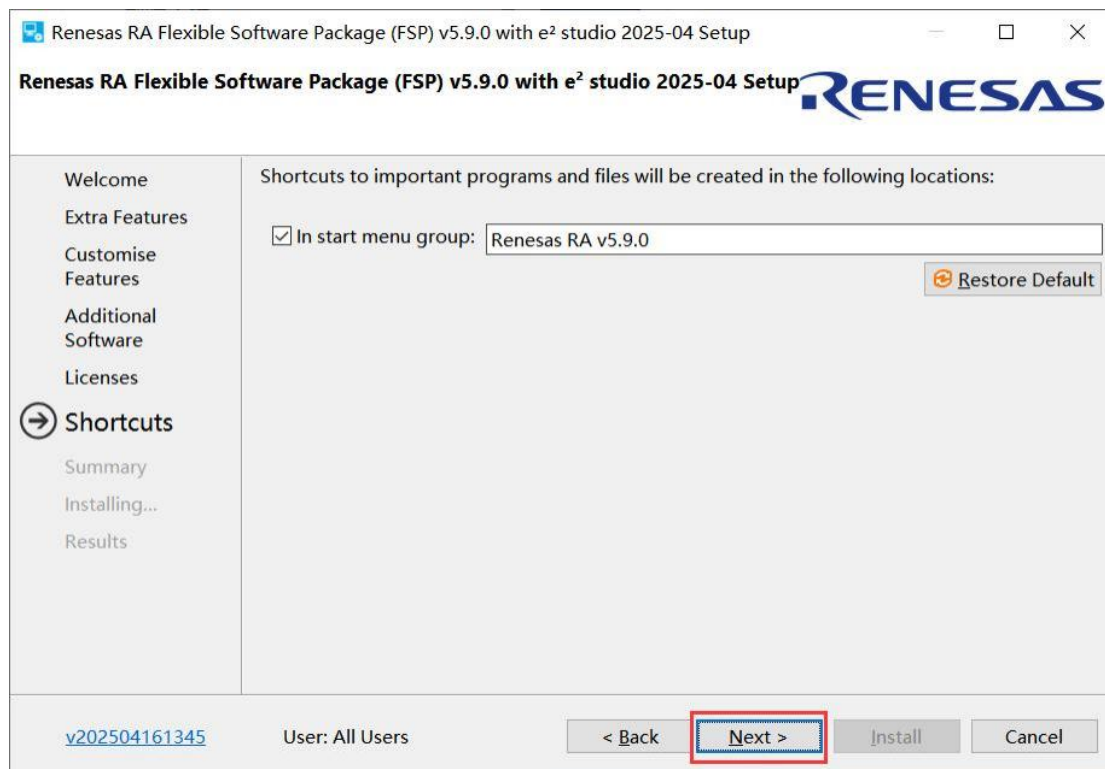
选择语言和一些组件，若不知道可以全选上：



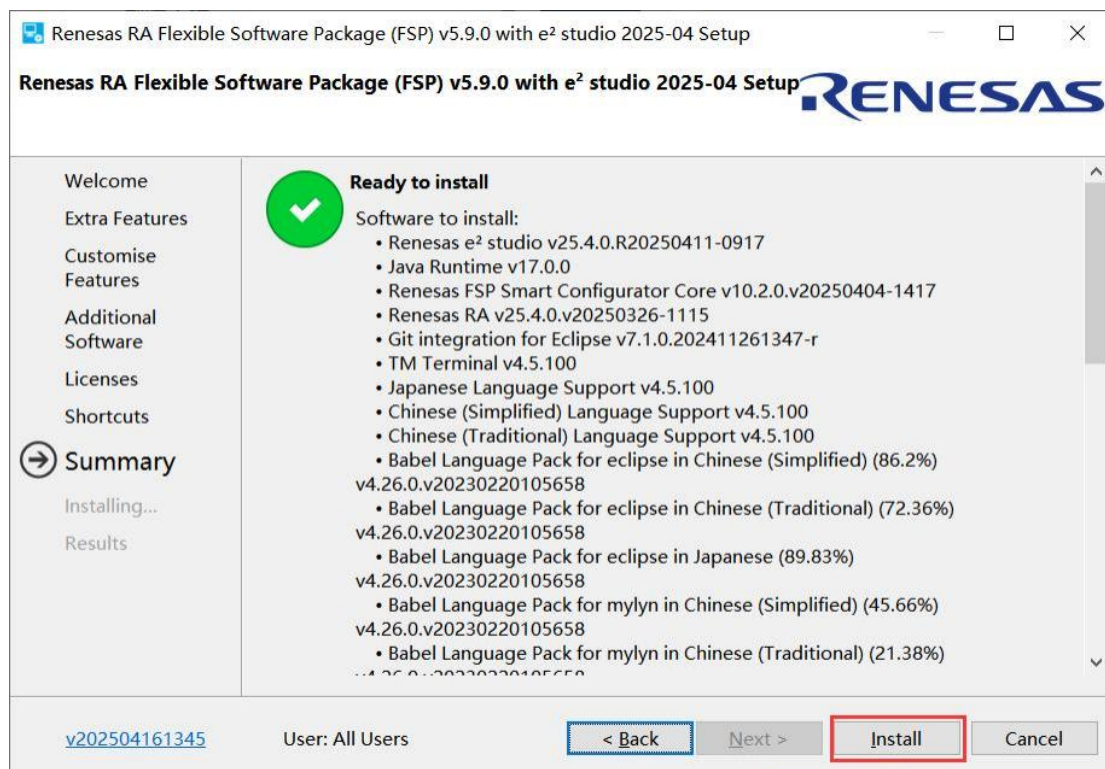


同意款项:

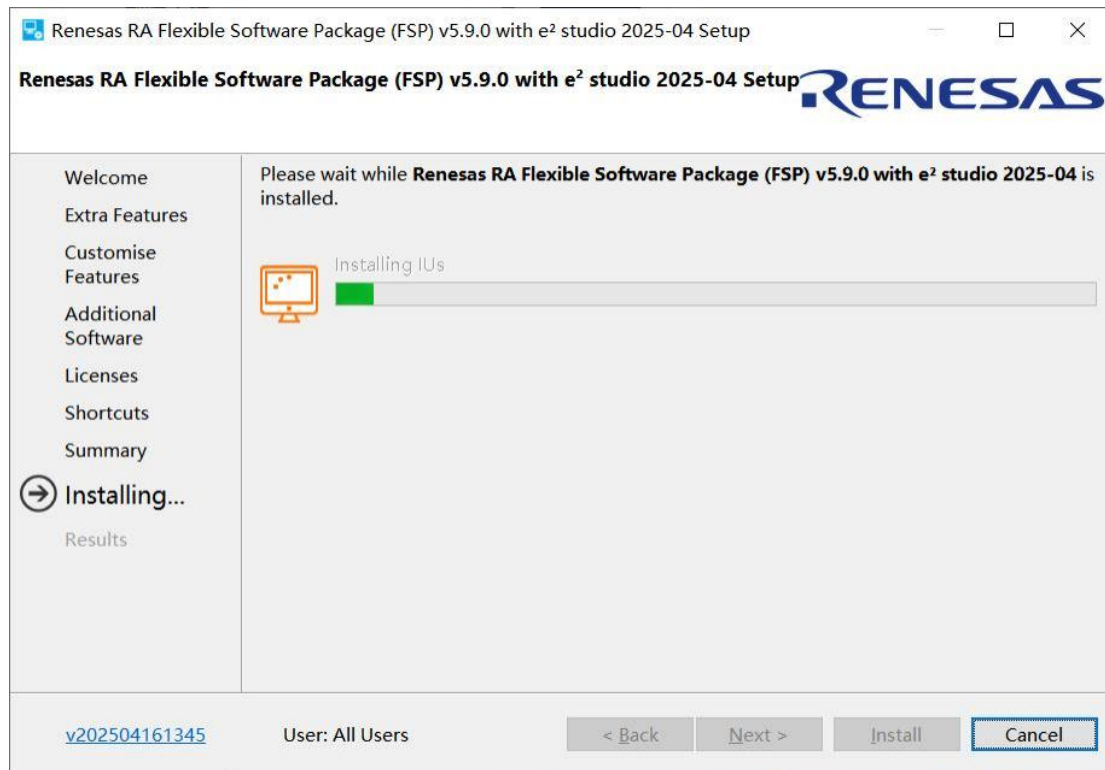




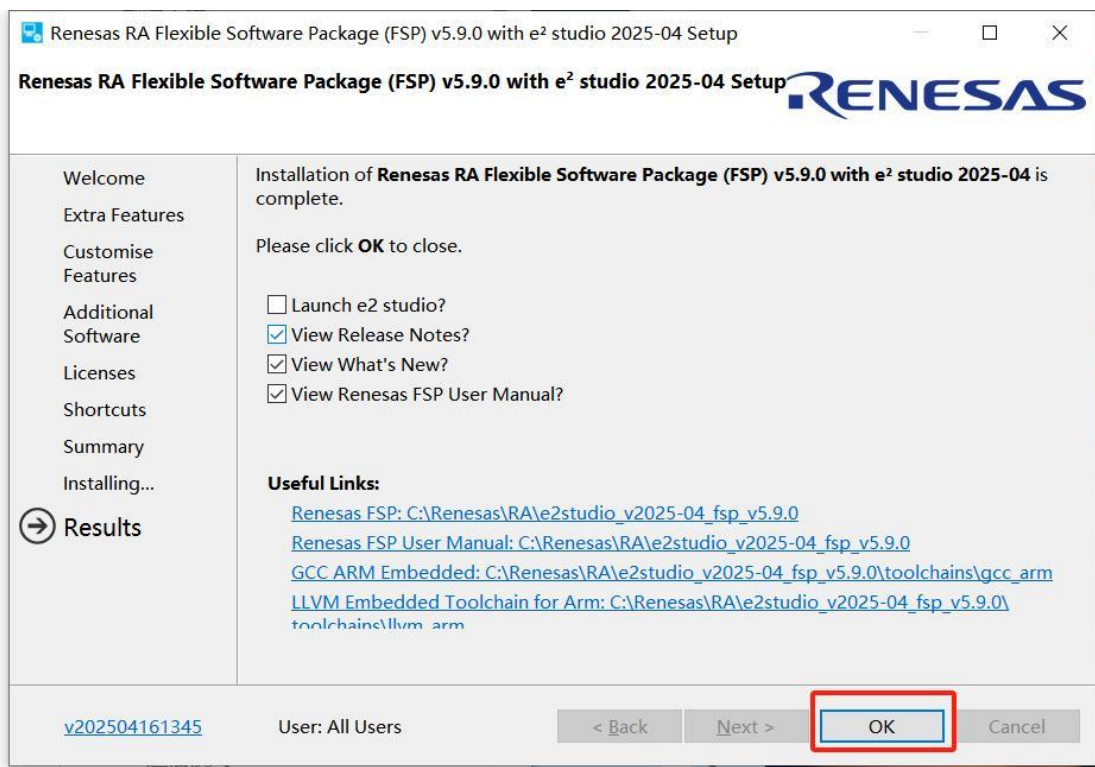
开始安装：



安装中：



安装完成:



project1: 开发板测试和 Programmer 烧录程序

1.对话 AI 导师

4. ****安装 CH340 驱动****: RFP 通过 USB 转 UART 与开发板通信。请在刚才下载的示例程序压缩包中找到 `CH341SER.EXE` 文件, 并运行安装。安装成功后, 你可以在电脑的设备管理器中看到 CH340 设备。
5. ****BOOT 设置****: 请将开发板上的 ****BOOT 开关拨到 ON 状态****。这个设置决定了 MCU 的工作模式。
6. ****烧录测试****:
 - * 打开 Renesas Flash Programmer。
 - * 点击 `File` -> `New Project...`。
 - * 在弹出的窗口中, Microcontroller 选择 `RA`。
 - * Project Name 和 Project Folder 可以随意命名。
 - * Tool 选择 `COM port`。
 - * 点击 `Connect` 连接开发板。
 - * 连接成功后, 切换到 `Operation` 选项卡。
 - * 在 `Program File` 处, 选择你刚刚解压的示例程序中 `Debug` 目录下的 `.hex` 或 `.srec` 文件。
 - * 点击 `Start` 进行烧录。

烧录完成后, ****请将 BOOT 开关拨回 OFF 状态, 然后重新给开发板上电****。观察开发板上的指示灯或者其他反馈, 如果一切正常, 说明你的开发板和烧录环境都

输入 '教我做机器马' 开始, AI将全程引导...

发送

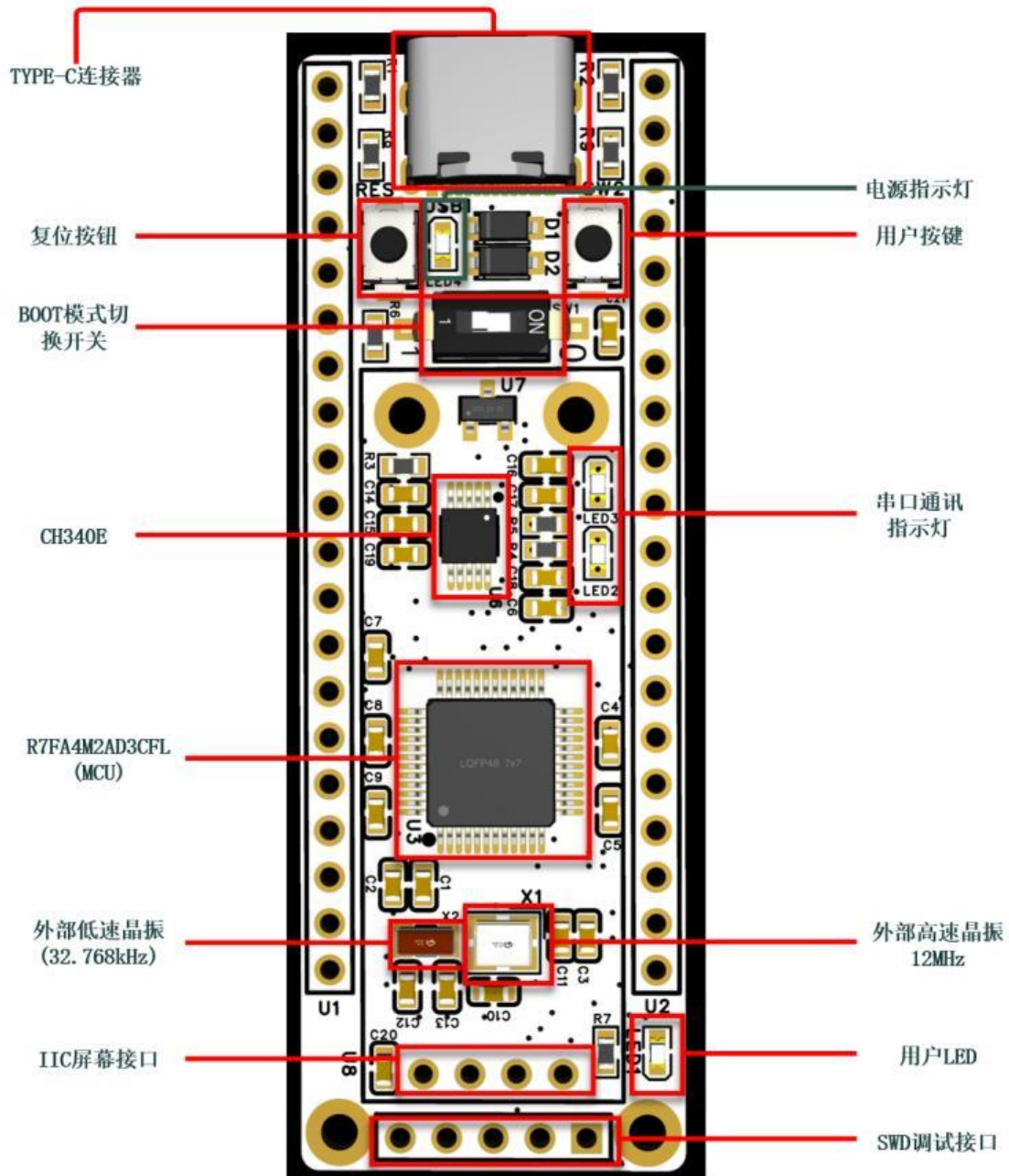
知识详解

2.概述

本章帮助用户在板卡到手后的“点亮”与基础功能验证: OLED 显示、LED 闪烁、按键切换与串口打印, 之后使用 RenesasFlashProgrammer (RFP) 给板子烧录代码。

3.硬件准备

本项目选用的主控为 R7FA4M2AD3CFL



4. 获取示例程序：

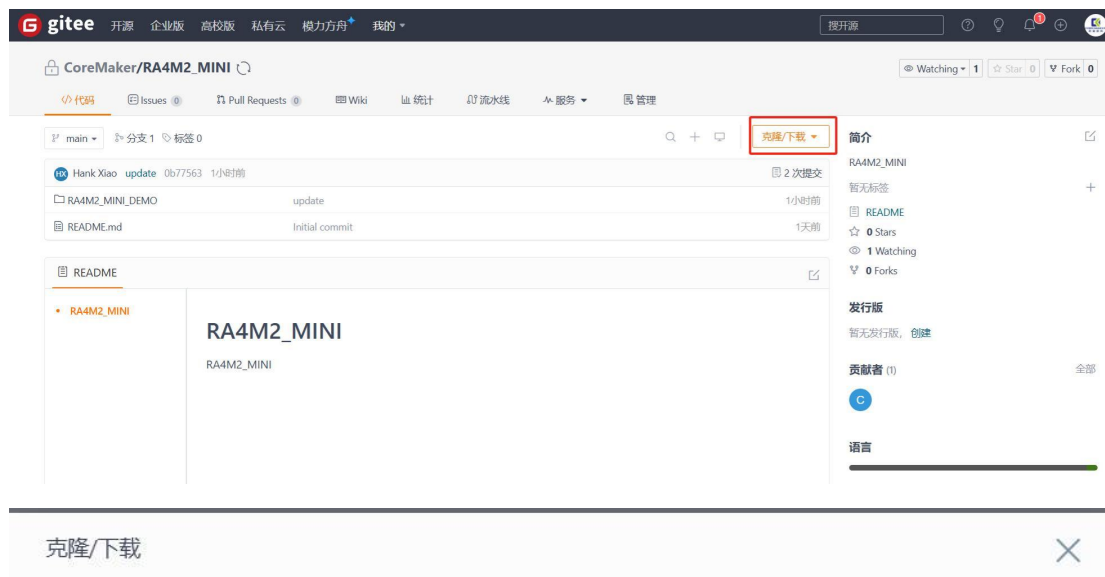
参考程序

https://github.com/CoreMaker-lab/RA4M2_MINI

https://gitee.com/CoreMaker/RA4M2_MINI

访问 **Gitee** 仓库：https://gitee.com/CoreMaker/RA4M2_MINI

点击克隆/下载→DownloadZIP，保存并解压。



克隆/下载

HTTPS SSH SVN SVN+SSH

下载ZIP

`https://gitee.com/CoreMaker/RA4M2_MINI.git`

提示

下载代码请复制以下命令到终端执行

```
git clone https://gitee.com/CoreMaker/RA4M2_MINI.git
```

为确保你提交的代码身份被 Gitee 正确识别，请执行以下命令完成配置

```
git config --global user.name 'CoreMaker'
git config --global user.email 'a845656974@qq.com'
```

使用 HTTPS 协议时，命令行会出现如下账号密码验证步骤。基于安全考虑，Gitee 建议 [配置并使用私人令牌](#) 替代登录密码进行克隆、推送等操作

Username for 'https://gitee.com': CoreMaker
Password for 'https://CoreMaker@gitee.com': 私人令牌

5.Programmer 理解与应用

RenesasFlashProgrammer 是一款支持开发和批量生产阶段写入的软件，具有简单的 GUI，即使是初学者也能简单操作进行写入。

运行环境支持 Windows、Linux、macOS。

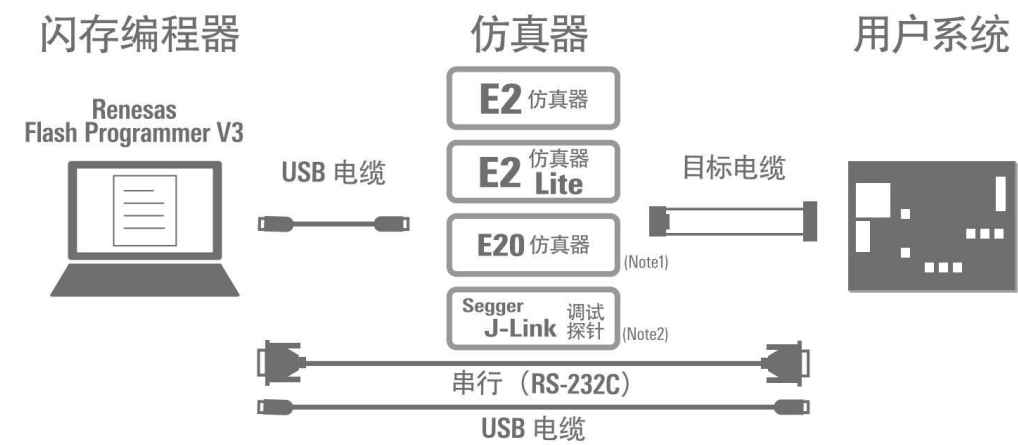
这里使用 RenesasFlashProgrammer 软件进行下载。

下载连接：<https://www.renesas.cn/zh/software-tool/renesas-flash-programmer-programming-gui>

5.1 主要特性

- (1) 支持的编程文件格式：IntelHEX（用于 HEX 和 HCUHEX 文件）、MotorolaS（用于 HEX 和 HCUHEX 文件）、二进制、RPI（用于 RPI 文件）
- (2) 通信接口支持 USB、CSI、UART、FINE、SWD
- (3) 校验计算
- (4) 空白检查、擦除、写入、验证、读取、校验和、闪存选项的设置和获取
- (5) 顺序执行多个命令（用于擦除、编程、验证等）
- (6) 同时选择多个编程文件

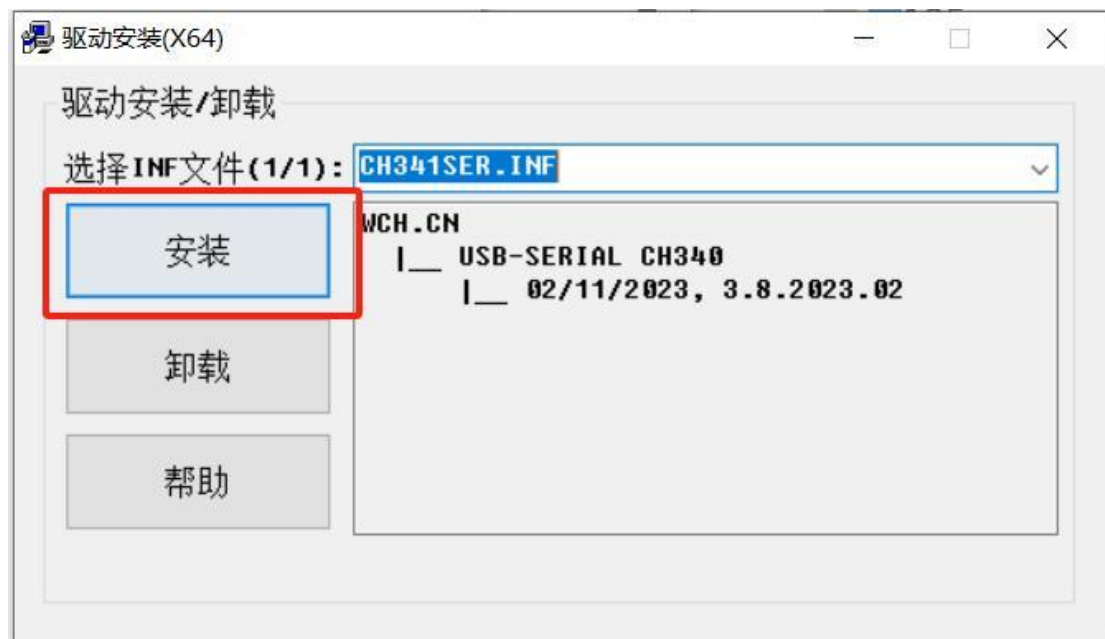
5.2 系统配置



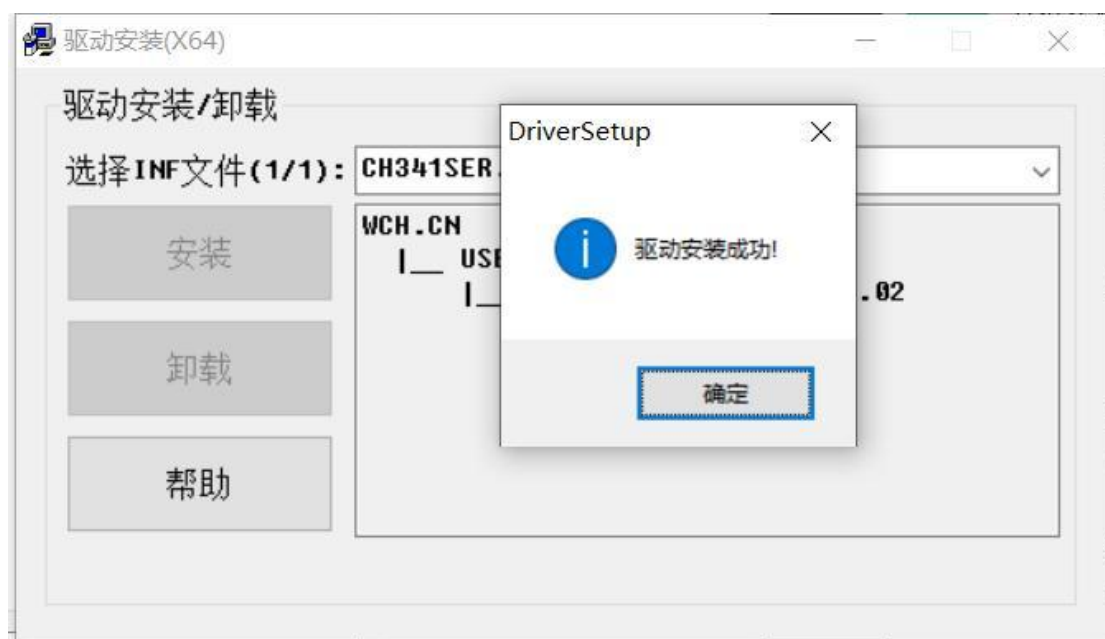
5.3 安装 CH340 驱动

它把电脑的 USB 信号转换成 UART（TTL 串口）信号，让开发板或模块能把日志、调试信息通过虚拟 COM 口发给电脑，也支持电脑向板子发送控制指令。没有驱动，操作系统无法识别这颗芯片，设备管理器里只会显示“未知设备”或带叹号的 USB 设备。





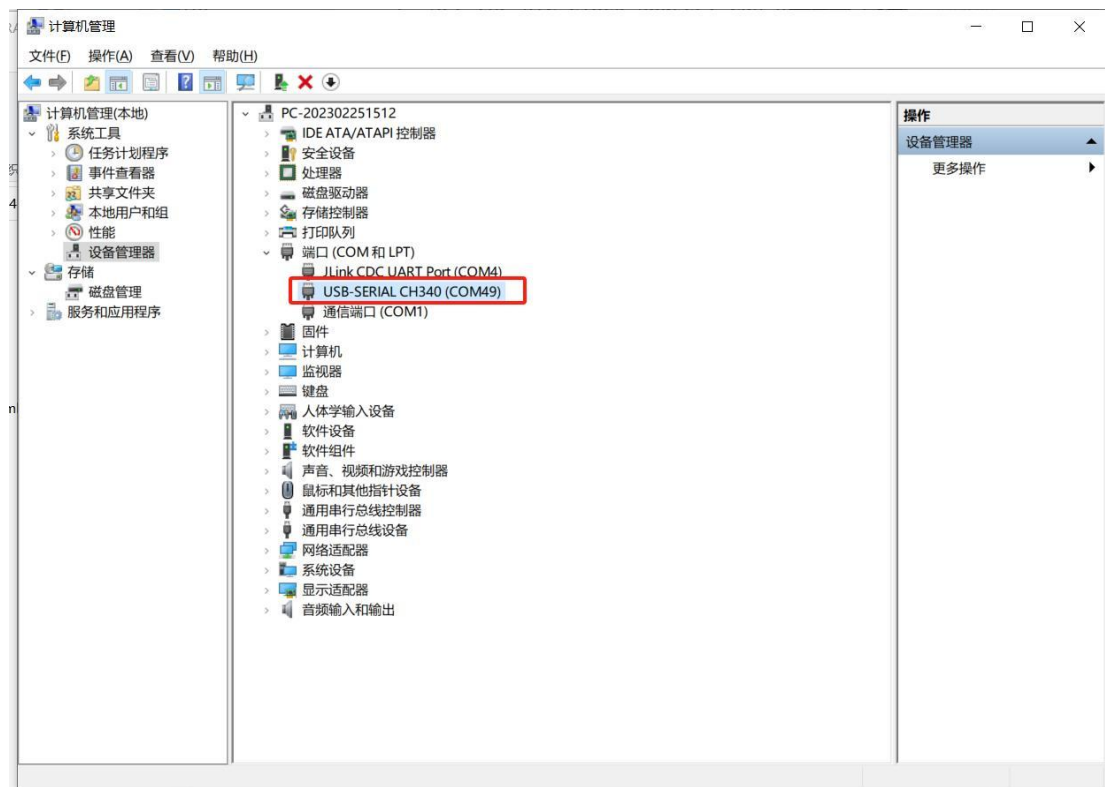
安装成功如下所示：



打开我的电脑的管理：



通过 USB 转 TYPE-C 线接入开发板，可以在设备管理器中查看到对应设备：



5.4 BOOT 设置

(1) 操作模式的切换取决于复位过程中的 MD 引脚状态：

MD=1：复位后进入单片模式。

MD=0: 复位后进入 SCI 引导模式。

3. Operating Modes

3.1 Overview

Table 3.1 shows the selection of operating modes by the mode-setting pin. For details, see [section 3.2. Details of Operating Modes](#). Operation starts with the on-chip flash memory enabled, regardless of the mode in which operation started.

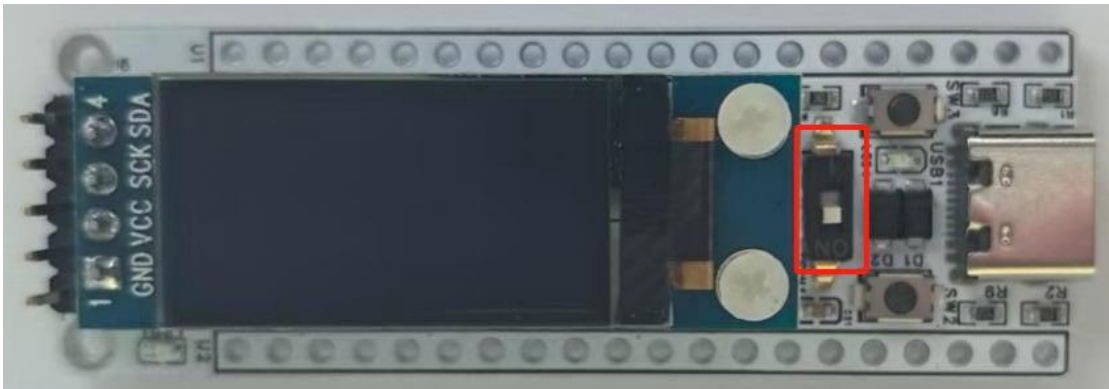
Table 3.1 Selection of operating modes by the mode-setting pin

Mode-setting pin (MD)	Operating mode	On-chip Flash
1	Single-chip mode	Enable
0	SCI / USB boot mode	Enable

(2) boot 模式可以通过拨码进行设置



(3) 切换如下模式进入烧录



5.5 串口烧录

串行编程模式包括：

通过 SCI9 的引导模式（BootmodewithSCI9）

板载已经将 P109,P110 给连接出来

44.13 Boot Mode

There are two serial programming modes; the boot mode (for the SCI interface) with SCI9 and the boot mode (for the USB interface) with USBFS. The available interfaces and connection times to the tool vary depending on the clock source connected to the MCU. Table 44.27 lists the I/O pins used in boot mode. Table 44.28 lists the available communication interface and the connection time depending on the clock sources in the boot mode.

Table 44.27 I/O pins used in boot mode

Pin name	I/O	Mode to be used	Use
MD	Input	Boot mode (for the SCI interface) Boot mode (for the USB interface)	Selection of operating mode
P110/RXD9	Input	Boot mode (for the SCI interface)	For host communication (to receive data through SCI)
P109/TXD9	Output		For host communication (to transmit data through SCI)
USB_DP, USB_DM	I/O	Boot mode (for the USB interface)	Data input/output of USB
USB_VBUS	Input		Detection of connection and disconnection of USB cables

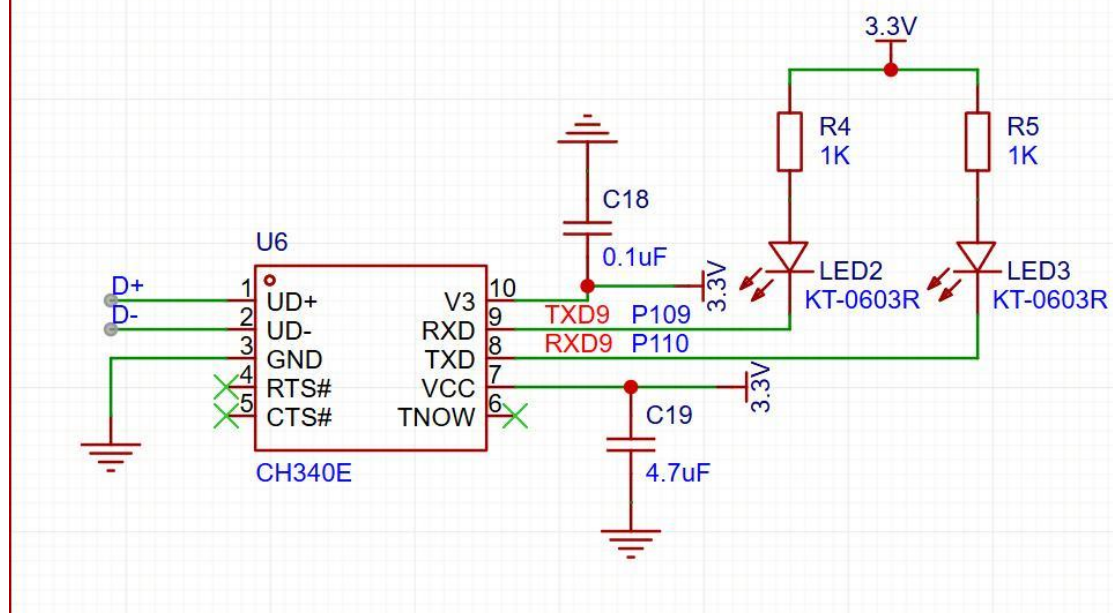
Table 44.28 Tool connection time depending on the clock source

Main clock oscillator	Sub-clock oscillator	Available interface	Tool connection time*2
Connected	Don't care	SCI/USB	Up to 1 second
Unconnected	Connected*1	SCI/USB	Up to 2 seconds
Unconnected	Unconnected	SCI	Up to 3 seconds

Note 1. The drive capability of the sub-clock oscillator is set to standard by SOMCR.SODRV bit. Note that if you use the crystal resonator corresponding to the low drive capability on your board, the crystal resonator may not oscillate in the boot mode.

Note 2. Tool connection time means that it takes for communication to be established between MCU and the host. For details, see the boot firmware application note.

串口CH340E

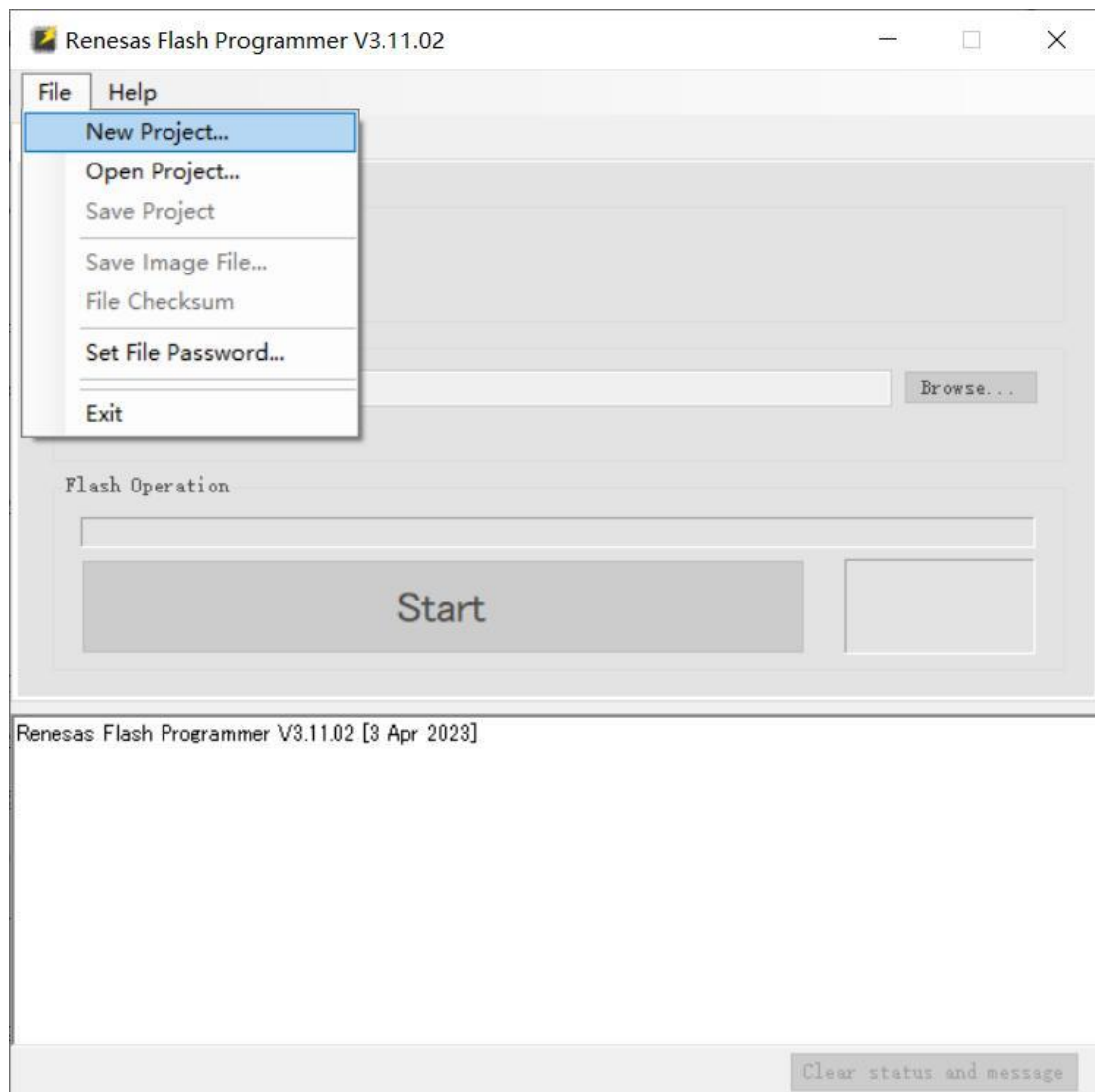


6. 烧录步骤

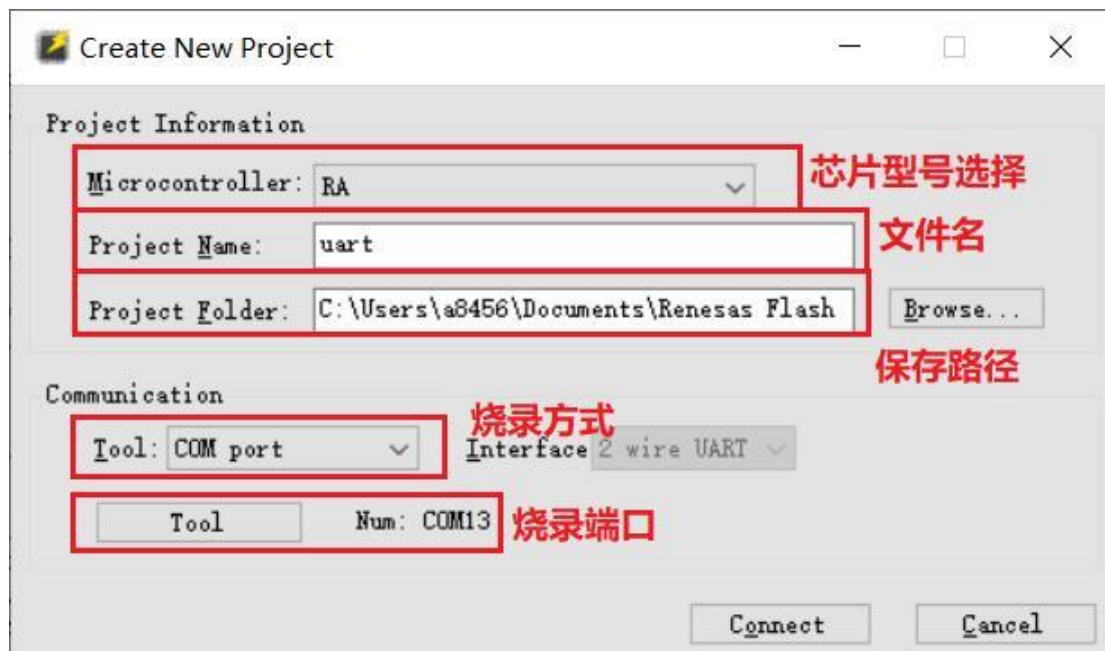
6.1 打开 RenesasFlashProgrammer



6.2 新建工程

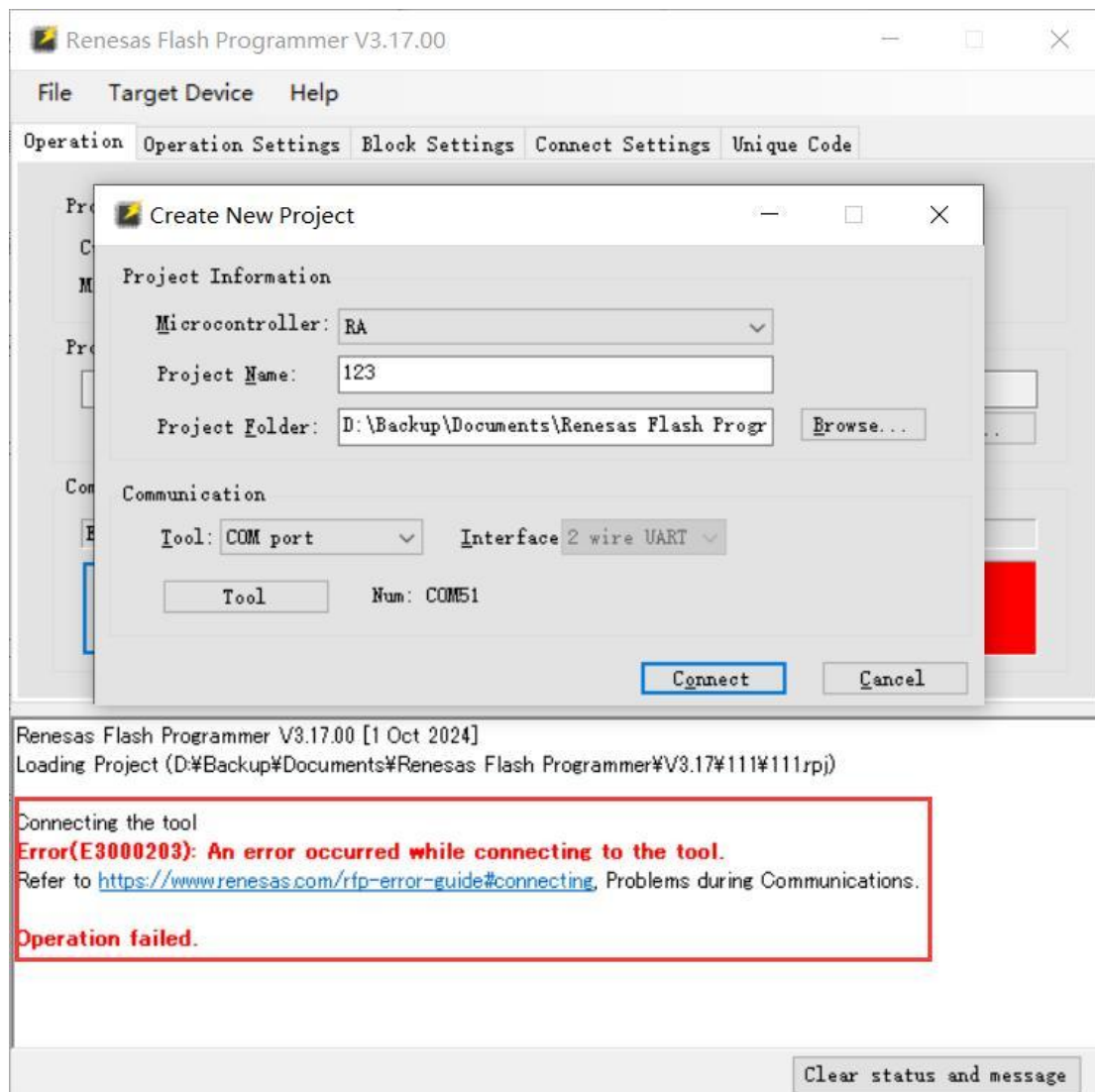


6.3 烧录设置



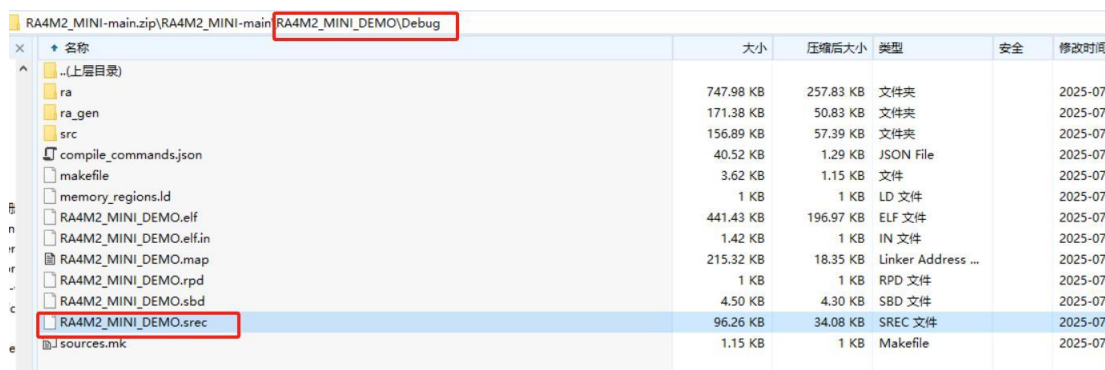
6.4 报错解决方案

若出现下列报错,需要按一下复位按键或者断电在上电,在重新设置即可。



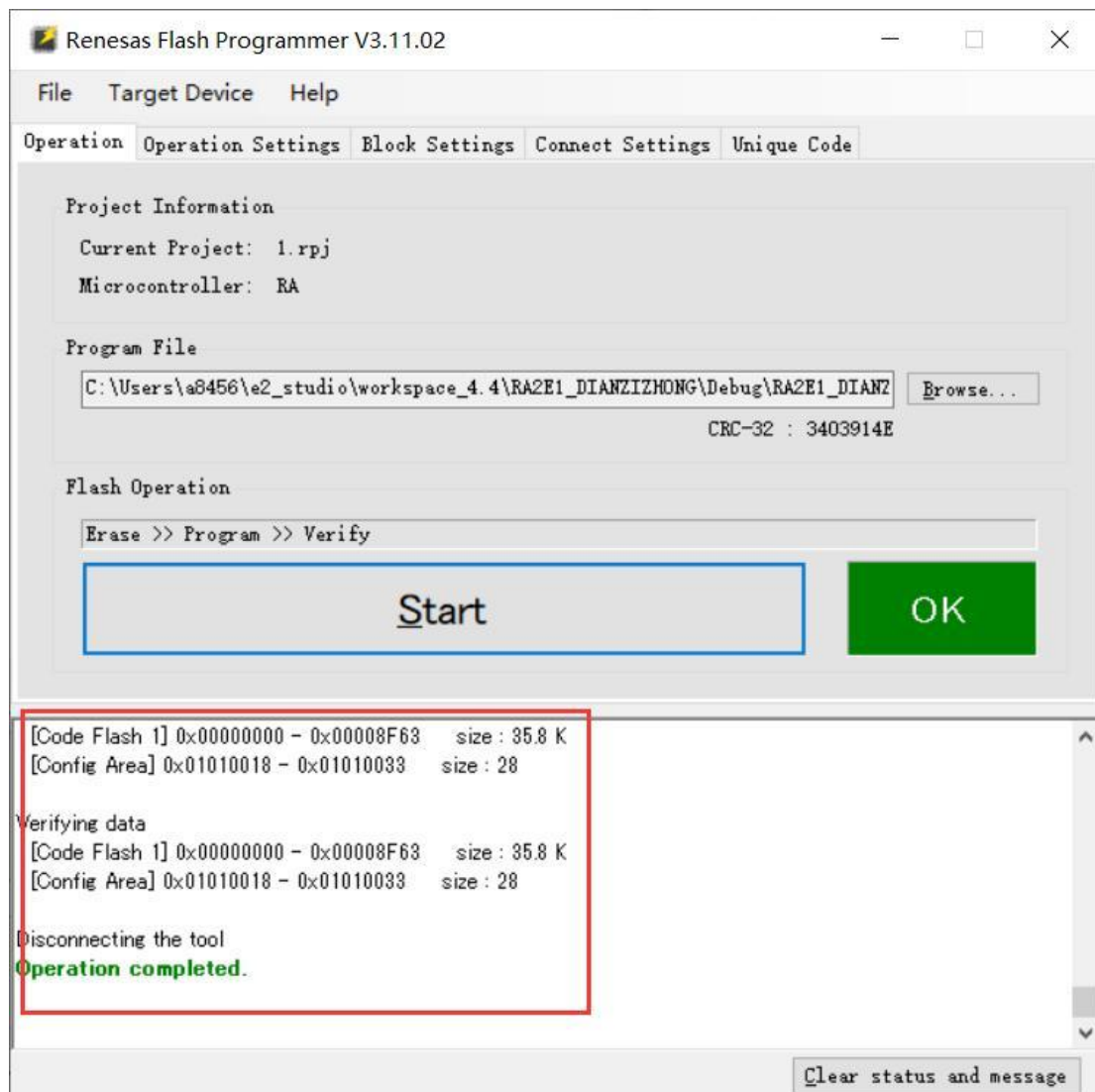
6.5 文件选择

烧录文件可以选 hex，也可以选 srec，选择刚刚下载文件中的 RA4M2MINIDEMO.srec。



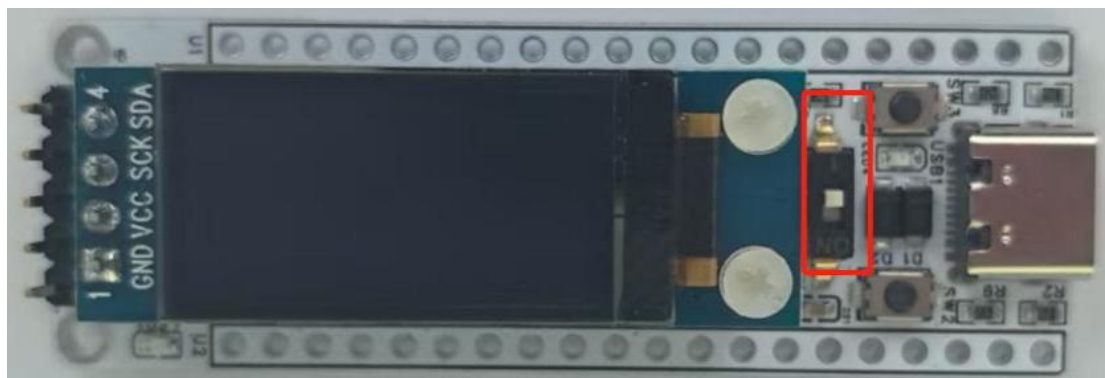
6.6 烧录成功

烧录成功如下所示。



6.7 重新上电

之后给板子断电，拨回如下状态，在进行上电。



显示如下图所示。



Project2: AI 导师获取与应用

1.获取 AI 导师

打开我们提供的资料包，在其中找到 AI 导师文件夹，双击打开其中的 AI 导师，此软件基于 gemini-2.5-flash-lite，将带领大家在一歩歩完成蓝牙小马项目，并且真正意义上实现零代码编写！

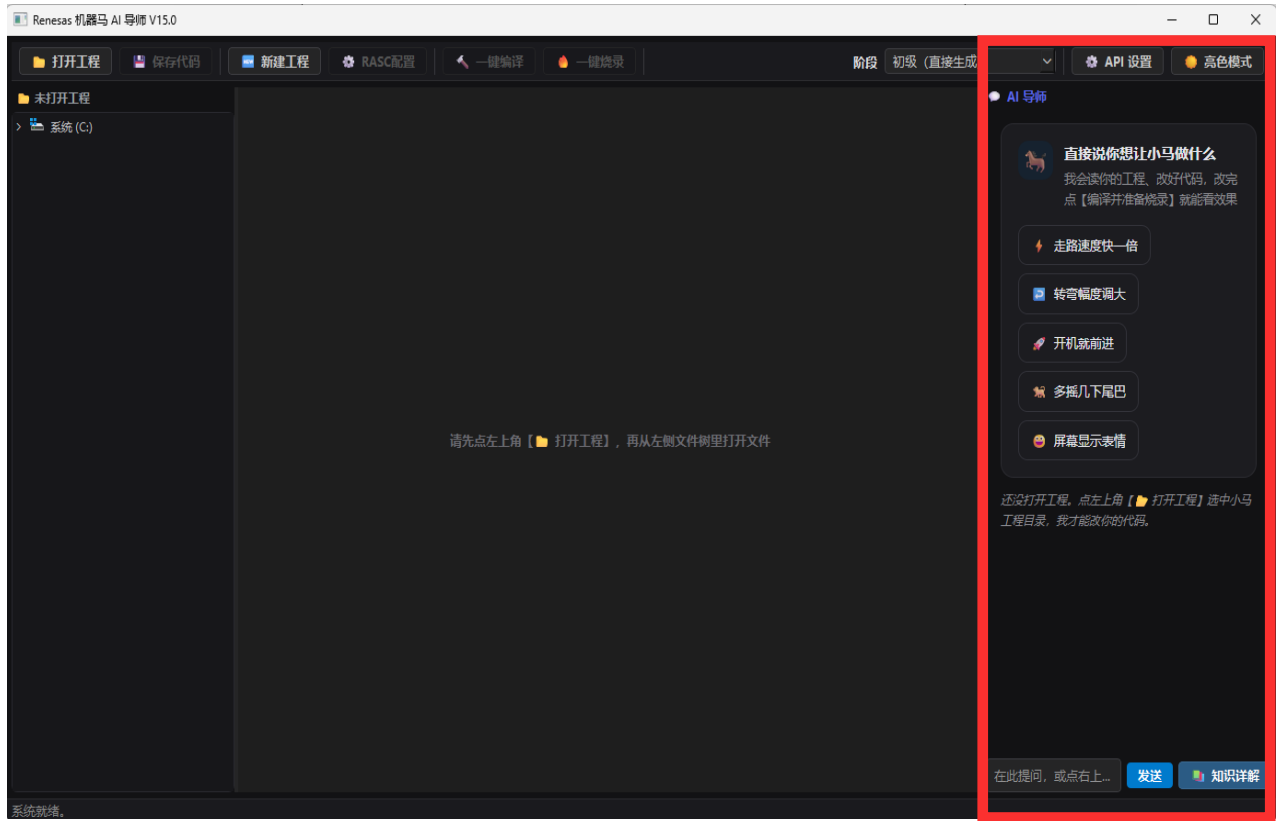
 RenesasHorseTutor.exe	2026/8/14 12:40	应用程序	10,152 KB
---	-----------------	------	-----------

2.了解 AI 导师

AI 导师主要具有以下功能：

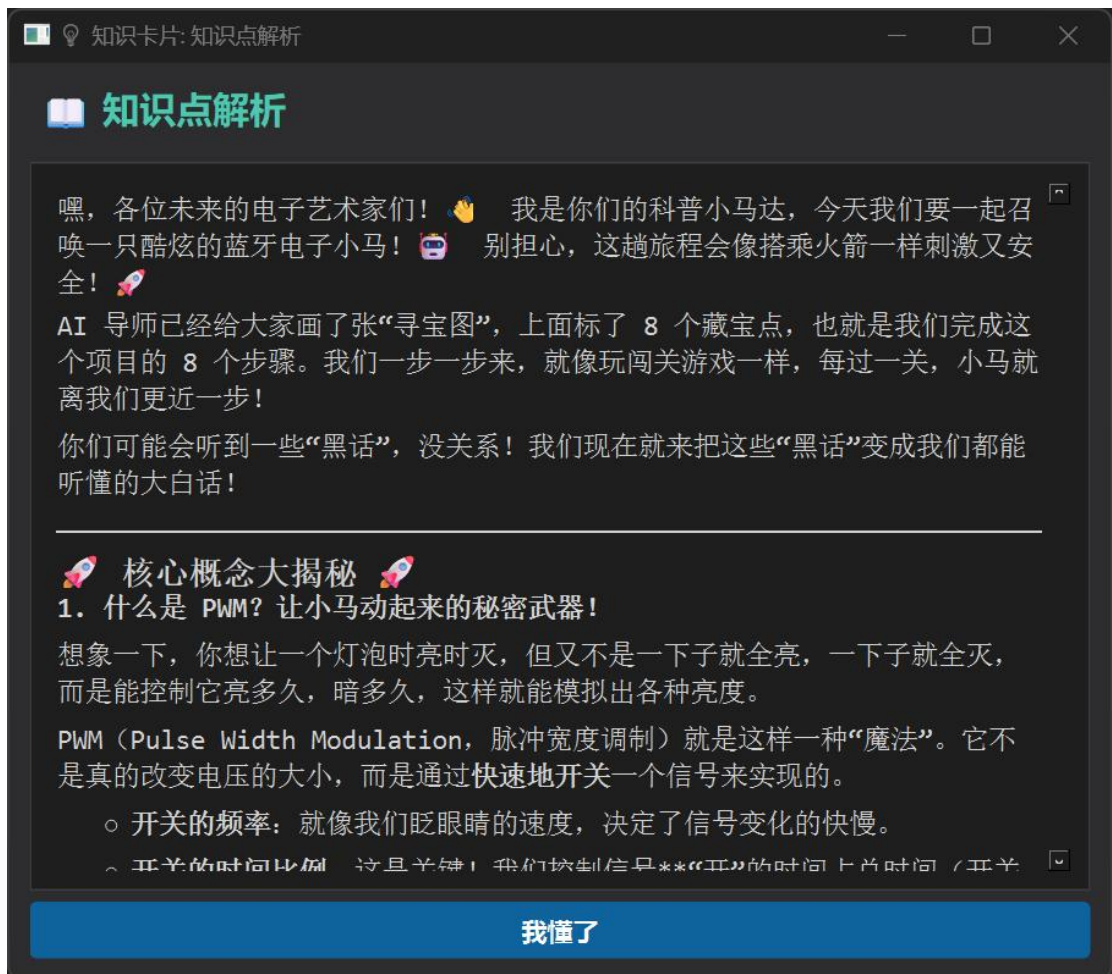
(1) 教学对话

在右侧的代码对话框中，我们可以与 AI 导师对话，在他的指引下去完成项目



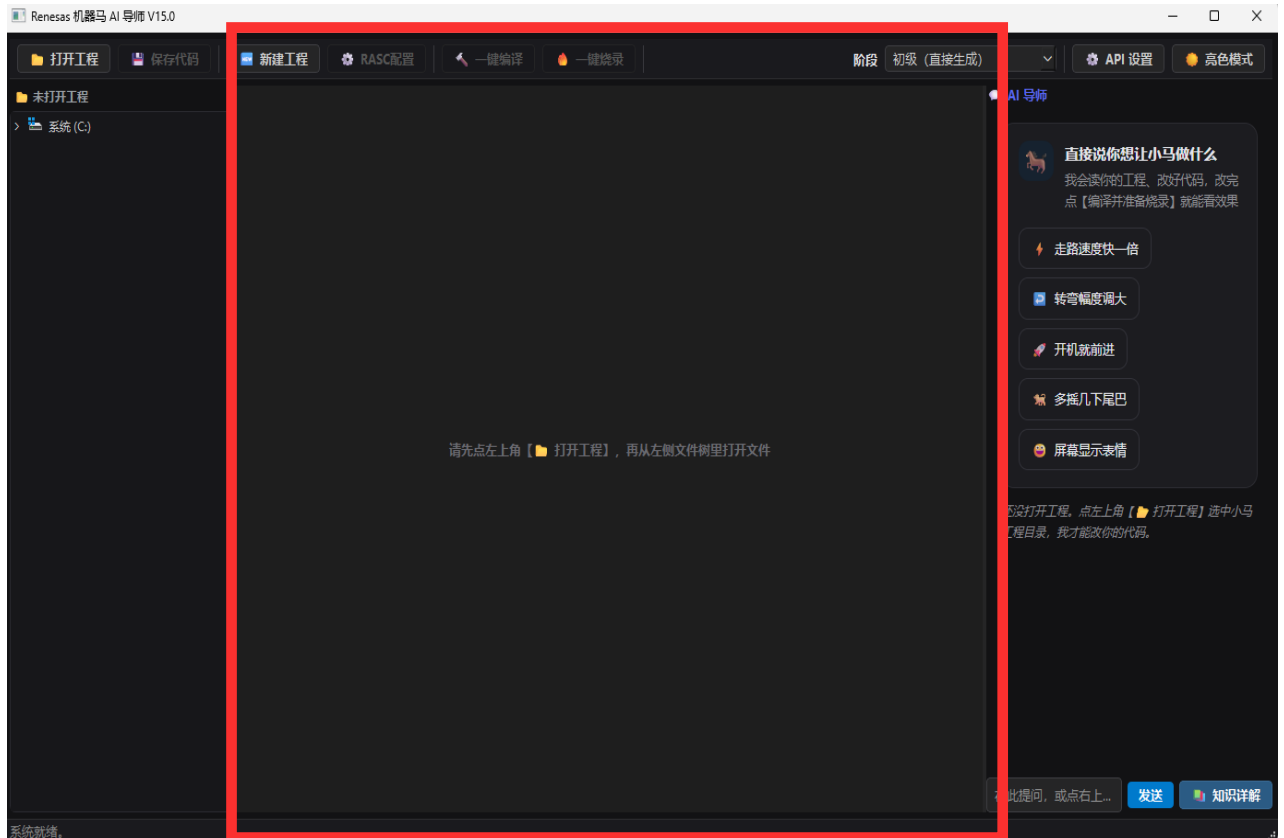
(2) 知识详解

当对某一步操作感到困惑，想要知其所以然时，我们点击页面右下角该模块，即可呈现出本步骤中涉及到的专业知识，帮助大家理解。



(3) 代码生成

由于本项目的目的是实现零代码编写，让萌新也能制作自己的蓝牙小马，所以在涉及到代码编写的步骤时，AI 导师将把需要的代码书写在左侧代码生成栏中，大家只需要复制到自己的工程中即可。



Project3：AI 导师辅助项目创建

1.项目介绍

进入 AI 导师后，它会为我们简单介绍项目的流程，大致了解后我们告诉 AI 导师我们准备好了。本教程中只展现关键步骤，具体的操作方法与原因将由 AI 导师提供，请于 AI 导师右侧的教学对话框中查看。

从 AI 导师口中得知该项目大致分为如下步骤：

- 1.工程创建
- 2.时钟树配置
- 3.调试引脚配置
- 4.定时器配置

5.串口配置

6.IC2 配置

7.编写 PWM 驱动

8.编写动作逻辑

7.导入 OLED 驱动

8.图片转格式与取模

9.编写表情逻辑

10.主程序编写

2.创建工程

2.1 对话 AI 导师

告诉 AI 导师我们准备好了，接下来根据 AI 导师的指引，开始创建工程：

1. 选取工作空间: 打开 e2 studio, 会跳出一个弹窗要求选取工作空间。第一次使用的同学可以先在 C 盘或 D 盘中创建一个文件夹, 起名 `e2s_workspace`, 这就是后续工程文件存放的地方。创建完成后回到此页面点击浏览, 选择刚创建的文件夹, 点击启动。
2. 新建项目: 左上角 文件(File) → 新建(New) → 瑞萨 C/C++ 项目 → Renesas RA。
3. 选取工程模板: 进入 Renesas RA 项目模板窗口后, 选择第一个 (Renesas RA C/C++ Project) 作为项目模板, 点击下一步。
4. 确定项目名称: 在 Project Name 里输入一个自己喜欢的名字, 比如 `Robohorse`。
5. 板卡型号与工具链的选择 (很重要!) :
 - Device 选 `R7FA4M2AD3CFL`
 - Debugger 选 `J-Link ARM`
 - Toolchains 选 `GNU ARM Embedded`
6. 以上配置完成后, 别的都选取默认配置就好了。

建好之后应该能看到 FSP 配置界面。如果找不到, 可以在左侧的项目资源管理器中重新找到 `configuration.xml` 并打开它。

⚠ 最后一步, 别漏了: 回到本软件, 点左上角【📁 打开工程】, 选中你刚建好的工程目录 (就是里面有 `configuration.xml` 的那一层)。
打开之后左侧会列出工程文件, 后面的编译、烧录、以及向我提问时我能不能看到你的代码, 都依赖这一步。

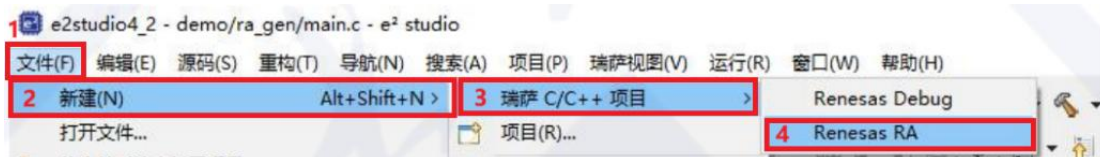
2.2 选取工作空间

按照指引完成后, 我们点开 e2studio 后将显示这样的界面:

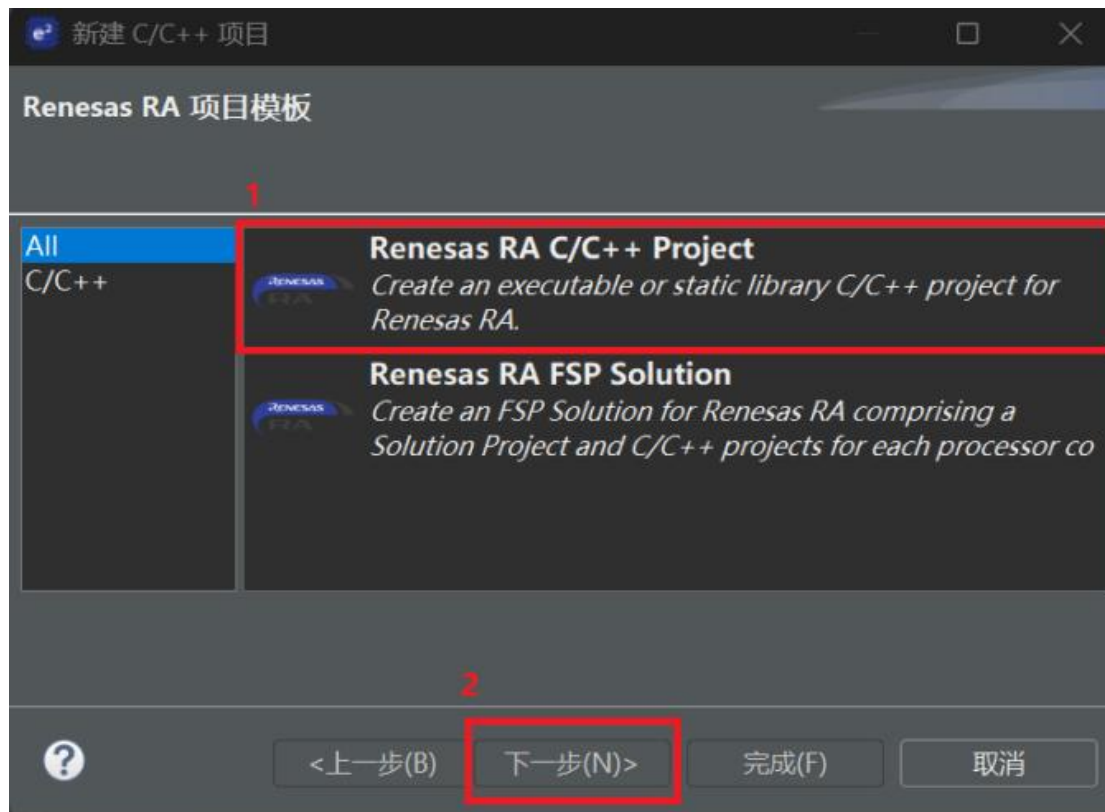


2.3 创建项目

告诉 AI 导师我们已经完成上一步配置, 接着在 AI 导师的指引下项目进行创建:



2.4 选取工程模板



2.5 确定项目名称



2.6 板卡型号与工具链选择

FSP Version: 5.9.0

FSP (Flexible Software Package) 是瑞萨 RA 系列 MCU 的官方软件开发套件，包含驱动库、中间件、配置工具等。这里选择 **5.9.0** 版本，决定了项目使用的驱动、HAL 库、中间件的版本，不同版本的 API、功能支持有差异，需和硬件、项目需求匹配。

Device: R7FA4M2AD3CFL

这是本次项目使用的具体 MCU 型号：

- 属于瑞萨 RA4M2 系列，Cortex-M33 内核，是项目的硬件核心。
- 选择该型号后，IDE 会自动匹配对应的引脚定义、时钟配置、外设驱动等资源。

GNU ARM Embedded

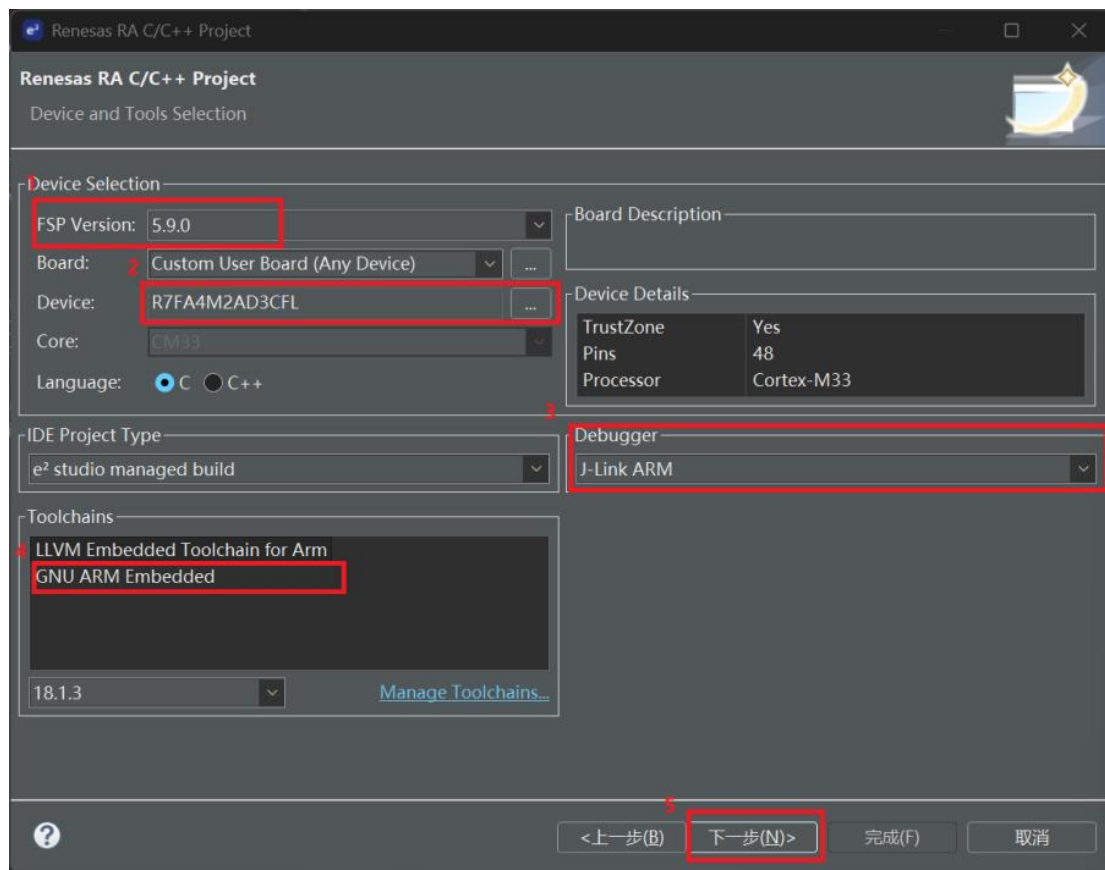
选择项目使用的编译工具链：

- GNU ARM Embedded 是开源免费的 ARM 架构编译器（基于 GCC），是瑞萨 RA 开发的主流工具链，兼容性好、社区支持完善。
- 上方的 LLVM 是另一种可选工具链，本项目选择了 GNU 版本。

Debugger: J-Link ARM

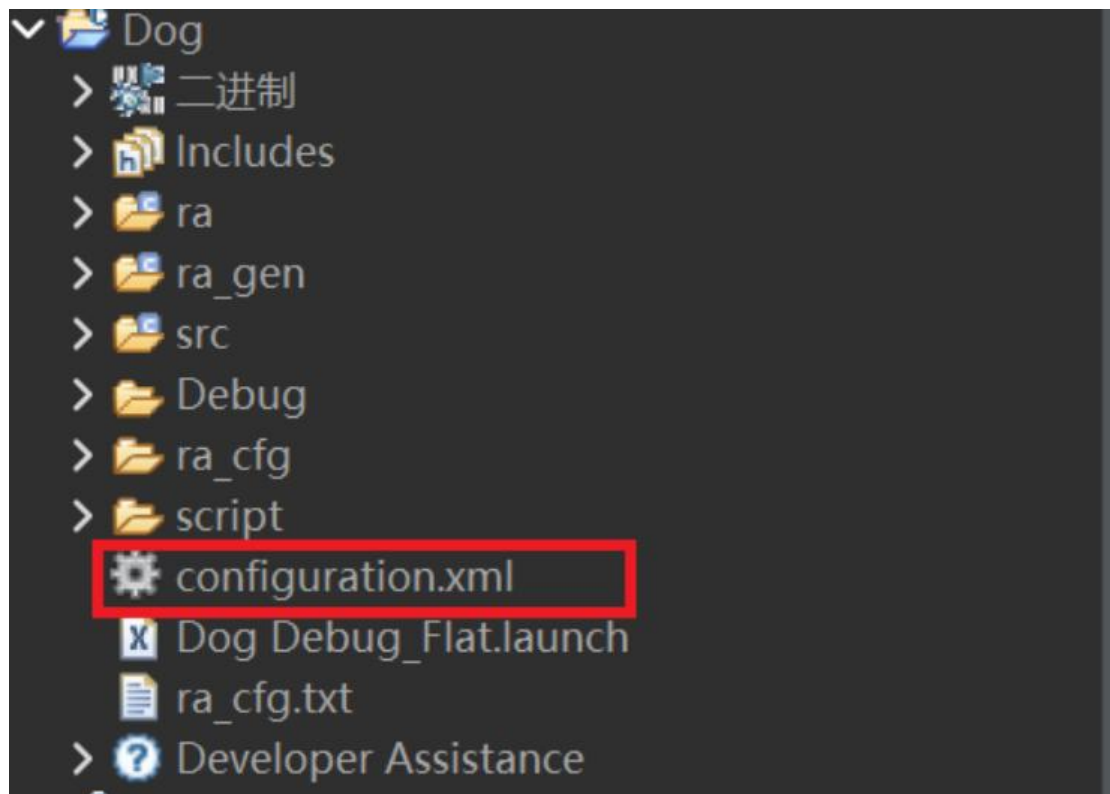
指定项目使用的调试器类型：

- 选择 **J-Link ARM**，表示使用 SEGGER J-Link 系列调试器进行程序下载、在线调试，是瑞萨开发中最常用的调试工具之一。
- 实际上我们采取的是串口下载，使用板载的 Type-c 接口



2.7 图形化配置页面位置

如果在创建工程完成后没有弹出图形化编辑页面，或者手误关掉了图形化配置页面，可在此处手动开启。



Project4: AI 导师辅助时钟树配置

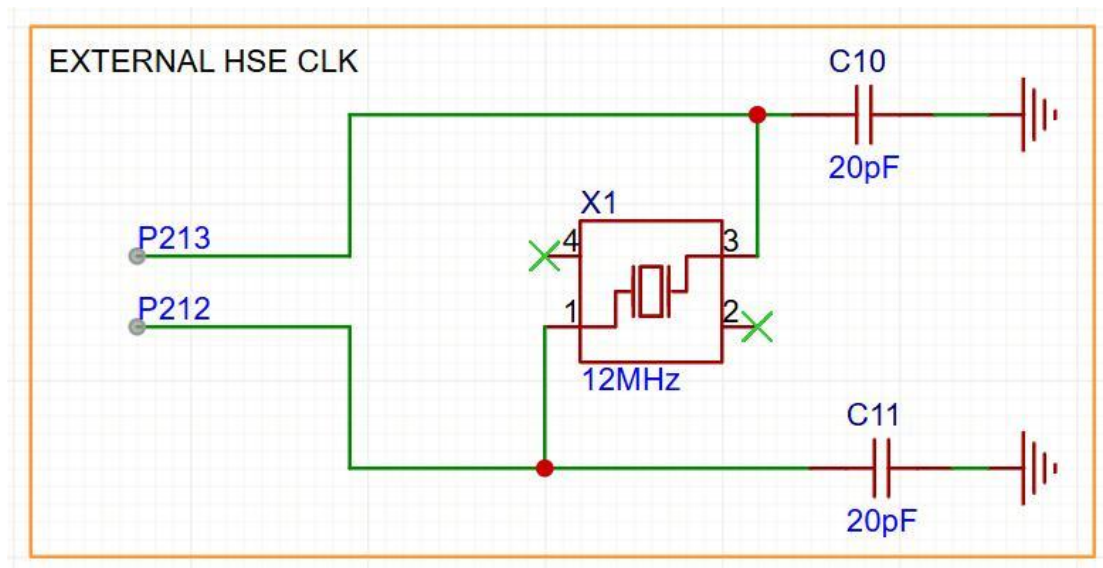
1. 了解时钟

外部高速晶振：开发板上板载外部高速晶振为 12M

时钟：时钟相当于单片机的“心脏”，当时钟正常工作，单片机才能正常运行

时钟树：此时外部晶振就相当于树根，从这个源头向外发散，可以把时钟分配给单片机的各个模块，在分配过程中可以进行分频、倍频操作

XTAL：即为 External Hse Clk，高速外部晶振的缩写



高速外部晶振原理图

2.对话 AI 导师

告诉 AI 导师我们已经完成上一步配置，在 AI 导师的指引下进行时钟树配置：

第 3 课 / 共 17 课 · 时钟树配置

1. 在图形化配置最下方的菜单栏中选择 **Clocks** 打开时钟树。
2. 将 XTAL **24MHz** 改为 **12MHz**。
3. 将 PLL **Div/3** 改为 **Div/2**。
4. 将 PLL Mul **x25.0** 改为 **x30.0**。

这三项都是在 e2 studio 默认值的基础上修改，其余保持默认。

Stacks Configuration

Threads

New Thread Remove

HAL/Common

g_ioport I/O Port (r_ioport)

Front_Foot Timer, General PWM (r_gpt)

R_Rear_Foot Timer, General PWM (r_gpt)

L_Rear_Foot Timer, General PWM (r_gpt)

Tail Timer, General PWM (r_gpt)

Blue_Tooth UART (r_sci_uart)

OLED I2C Master (r_iic_master)

Objects

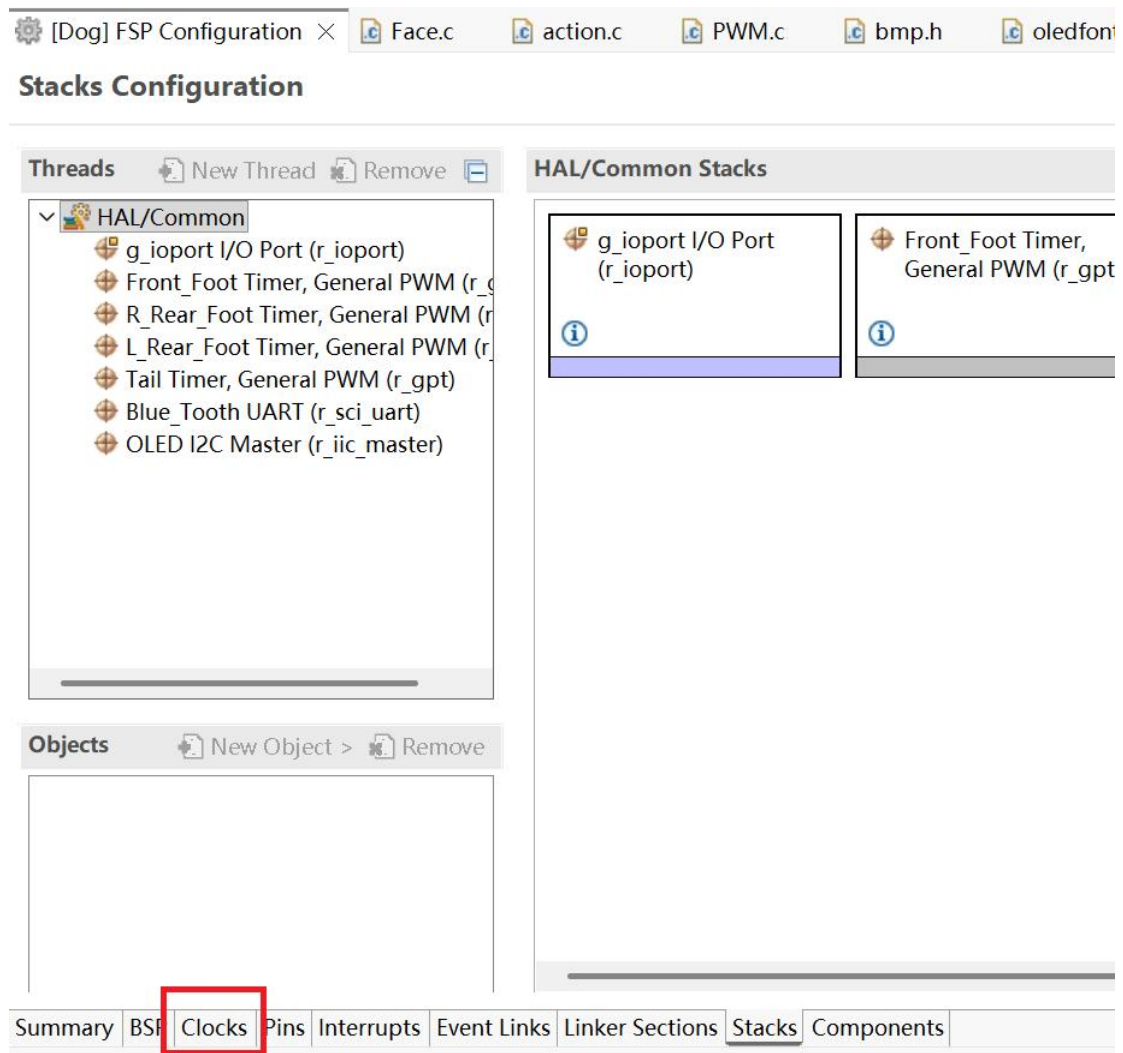
New Object > Remove

HAL/Common Stacks

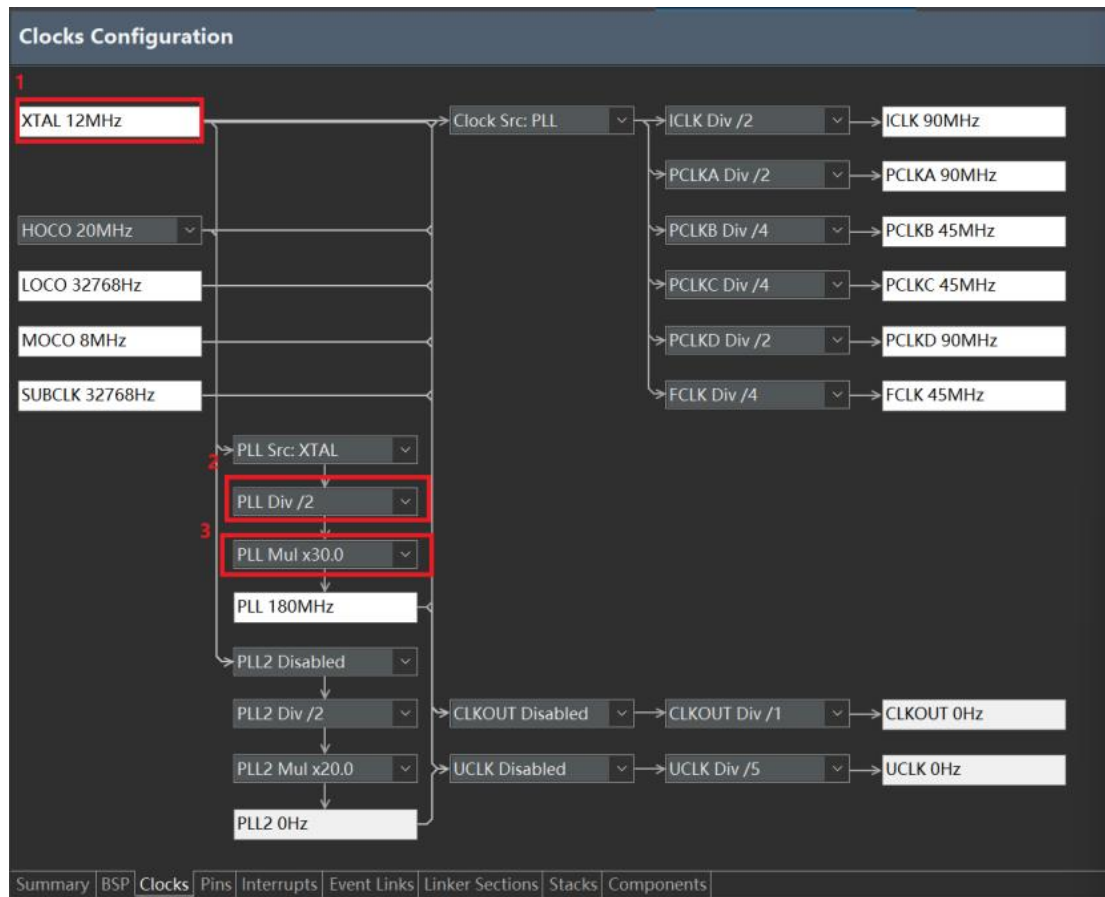
g_ioport I/O Port (r_ioport)

Front_Foot Timer, General PWM (r_gpt)

SummaryBSPIOClocksPinsInterruptsEvent LinksLinker SectionsStacksComponents



3.时钟树配置完成页面



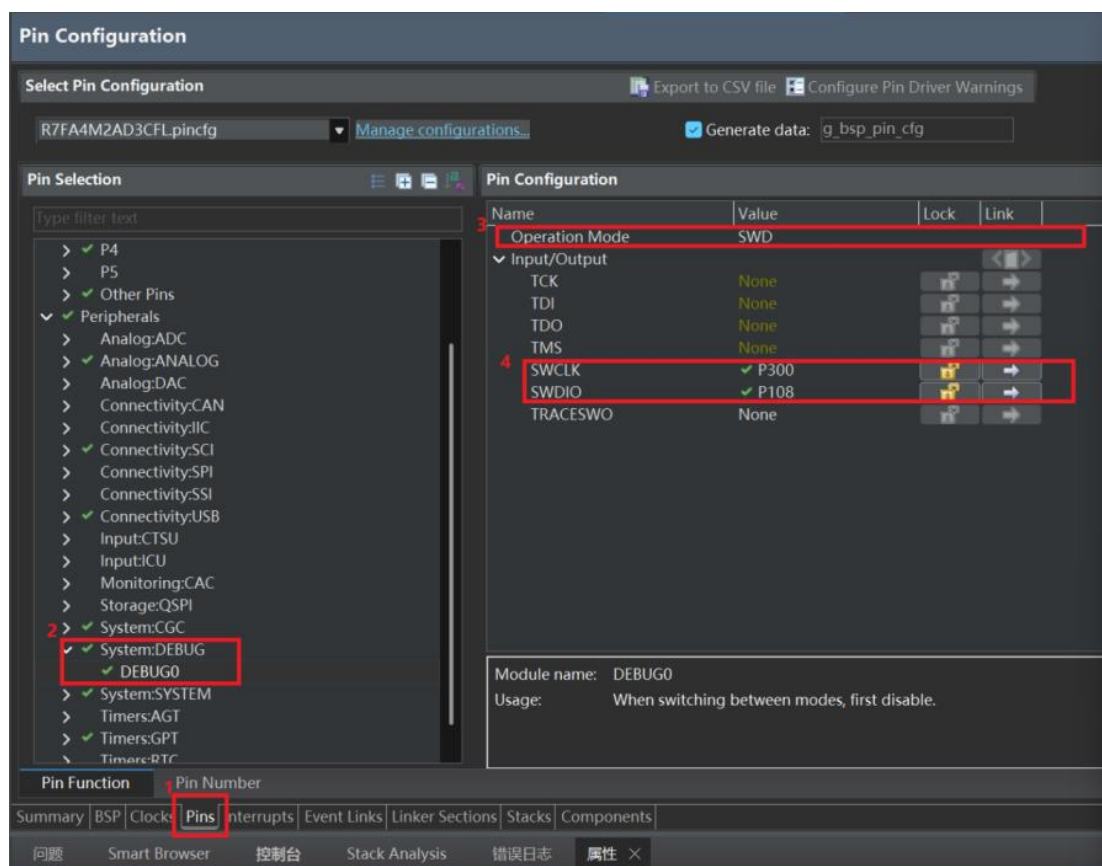
Project5: AI 辅助调试引脚配置

1.对话 AI 导师

告诉 AI 导师我们已经完成上一步配置，在 AI 导师的指引下进行调试引脚配置：

1. 点击最下方菜单栏中的 **Pins** 打开引脚配置界面。
2. 在左侧选择"系统调试"选项，将调试模式改为 **SWD (Serial Wire Debug)** (如果此处默认已经是 SWD，可以不改)。
3. 再将 **SWCLK** 和 **SWDIO** 分别配置为 **P300** 和 **P108**。

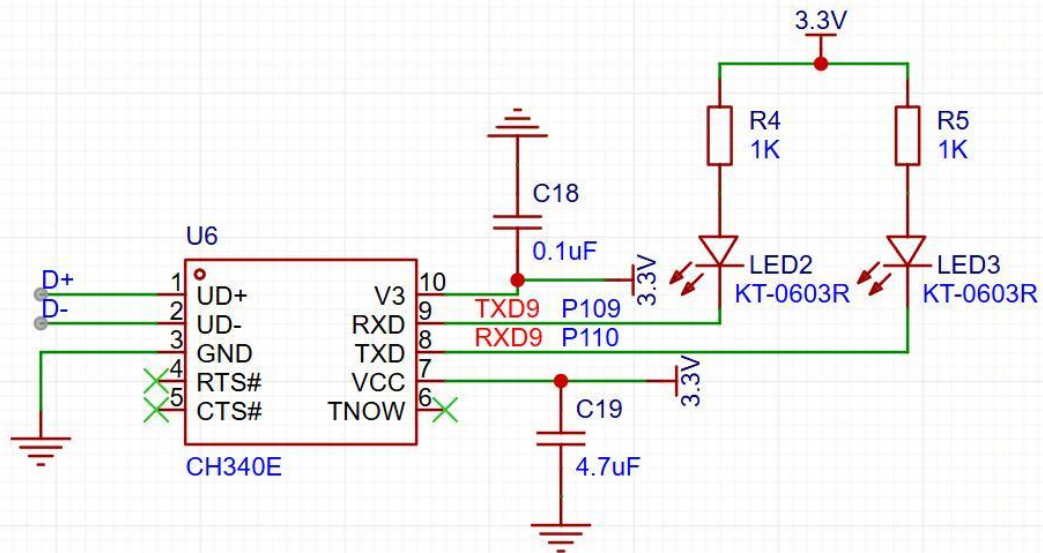
2. 调试引脚配置完成页面



3. 原理解释

由串口的原理图我们可以发现 CH340E 的 TX 与 RX 引脚分别与单片机的 P109 和 P110 引脚相连，为了使两者正常通信，所以我们选择配置调试引脚为 P109 和 P110

串口CH340E



Project6: AI 辅助定时器配置

1.了解定时器

GPT（通用 PWM 计时器）模块是一个高精度定时器，用于计数事件、测量外部输入信号、生成周期性中断，或输出周期性信号/PWM 信号到 GTIOC 引脚。它广泛应用于定时控制、信号测量、PWM 生成等应用，尤其适用于电机控制、LED 调光、PWM 伺服驱动和 ADC 触发等场景。

2.对话 AI 导师：

告诉 AI 导师我们已经完成上一步配置，在 AI 导师的指引下进行时器配置：

第 5 课 / 共 17 课 · 定时器 (PWM) 配置

1. 点击最下方菜单栏里的 **Stacks** 进入软件组件与驱动模块配置。
2. 点击右上角 **New Stack** → **Timers** → **Timer, General PWM (r_gpt)**。
3. 我们需要创建 4 个定时器模块，提供 5 路 PWM 输出，分别控制机器马的 4 条腿和尾巴（前腿模块用 GTIOC1A / GTIOC1B 两路，各驱动一条前腿）。每个模块的配置都一样，只有名字、通道和引脚不同。
4. 在 **Module xxx, General PWM(r_gpt)** 的属性里按下表配置：

通用设置（4 个模块都改成一样的）

- **General** 里：Mode = **Periodic**，Period = **50** Hz（注意单位选 Hz）
- **Output** 里：Duty Cycle Percent = **50**（先设 50%，后续在代码里动态修改占空比来控制动作）
- **Output** 里：GTIOCA Output Enabled = **True**
- **Output** 里：GTIOCA Stop Level = **Pin Level Low**

依次配置 4 个模块的 **Name / Channel / Pins**

- 模块1（前腿）：Name **Front_Foot** → Channel **1** → GTIOC1A: **P109**，GTIOC1B: **P104**
- 模块2（右后）：Name **R_Rear_Foot** → Channel **2** → GTIOC2A: **P103**，GTIOC2B: **None**
- 模块3（左后）：Name **L_Rear_Foot** → Channel **3** → GTIOC3A: **P111**，GTIOC3B: **None**
- 模块4（尾巴）：Name **Tail** → Channel **4** → GTIOC4A: **P302**，GTIOC4B: **None**

3. 创建组件

Stacks Configuration

Threads

New Thread Remove

HAL/Common

- g_ioport I/O Port (r_ioport)
- Front_Foot Timer, General PWM (r_gpt)
- R_Rear_Foot Timer, General PWM (r_gpt)
- L_Rear_Foot Timer, General PWM (r_gpt)
- Tail Timer, General PWM (r_gpt)
- Blue_Tooth UART (r_sci_uart)
- OLED I2C Master (r_iic_master)

Objects

New Object > Remove

HAL/Common Stacks

g_ioport I/O Port (r_ioport)

Front_Foot Timer, General PWM (r_gpt)

Summary

BSP

Clocks

Pins

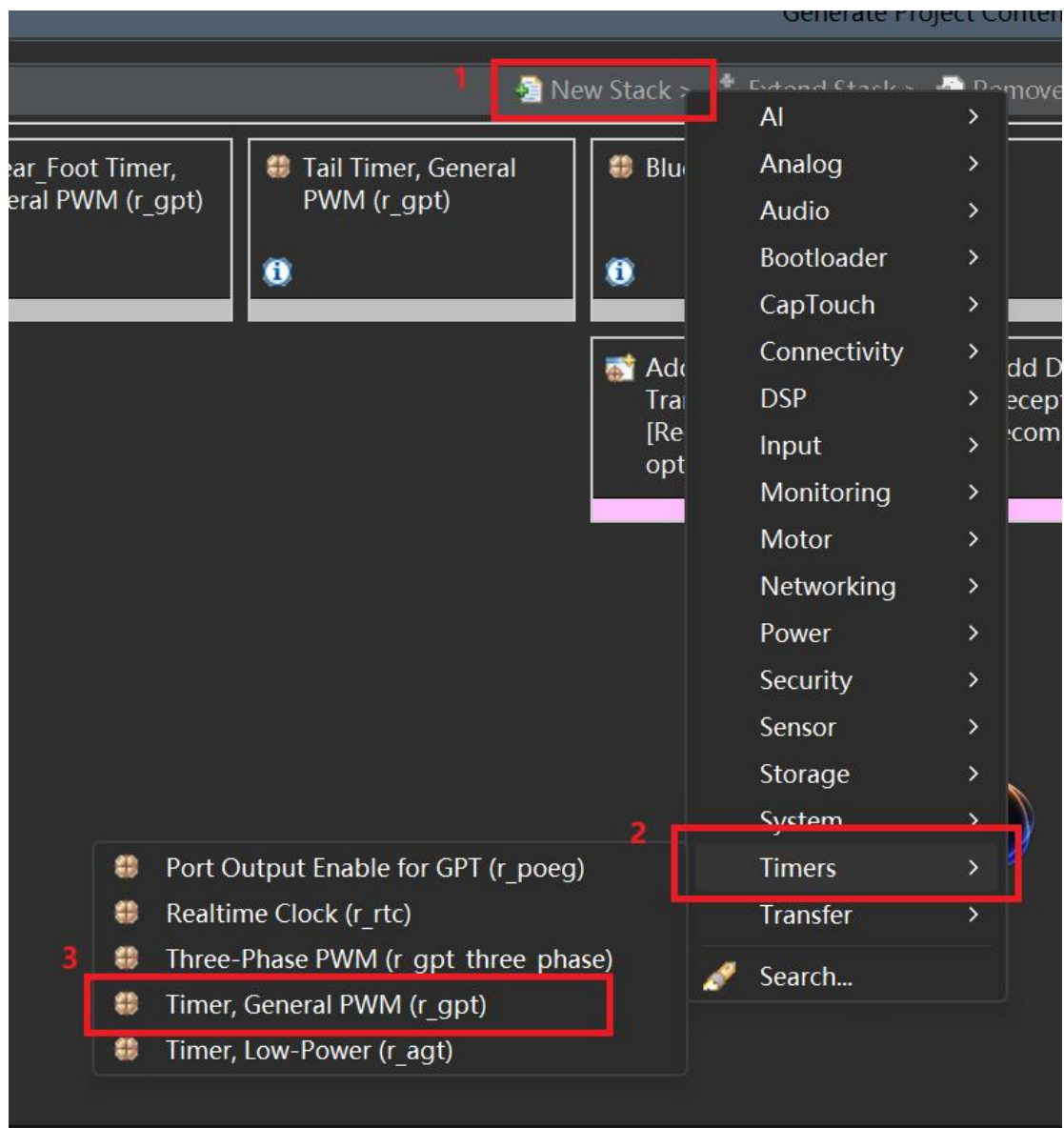
Interrupts

Event Links

Linker Sections

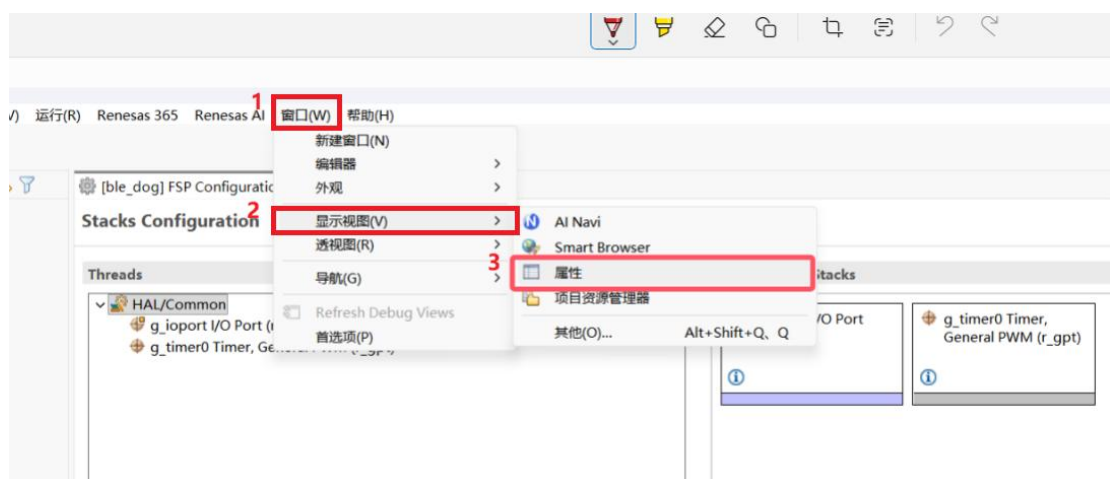
Stacks

Components



3.属性配置

若点击组件后面板上未自动出现属性页面，请按步骤打开属性页面：



此处仅展示一个定时器的配置，其余定时器配置请根据 AI 导师的指导进行配置。

问题	Smart Browser	控制台	Stack Analysis	错误日志	属性 ×
Front_Foot Timer, General PWM (r_gpt)					
Settings	属性				
API Info	值				
	Common				
	Parameter Checking				
	Default (BSP)				
	Pin Output Support				
	Enabled				
	Write Protect Enable				
	Disabled				
	Module Front_Foot Timer, General PWM (r_gpt)				
	General				
	Compare Match				
	Name				
	Front_Foot				
	Channel				
	1				
	Mode				
	Periodic				
	Period				
	50				
	Period Unit				
	Hertz				
	Output				
	Custom Waveform				
	Duty Cycle Percent (only applicable in PWM mode)				
	50				
	GTIOCA Output Enabled				
	True				
	GTIOCA Stop Level				
	Pin Level Low				
	GTIOCB Output Enabled				
	True				
	GTIOCB Stop Level				
	Pin Level Low				
	Input				
	Interrupts				
	Extra Features				
	Pins				
	GTIOC1A				
	P109				
	GTIOC1B				
	P104				

4.属性介绍

- PinOutputSupport: 设为 Enabled，允许 PWM 信号输出到外部引脚。
- Name: 这是模块的唯一标识符，用于代码中引用此定时器实例。
- Channel: 表示该定时器使用选中的通道进行计数和控制。
- Mode: 设为 Periodic，表示该定时器工作在周期模式，将在设定周期内重复运行。
- Period: 周期值设置为 1，即每个完整计数周期的时间间隔。
- PeriodUnit: 单位选择为 Kilohertz（千赫兹），表示输出信号的频率为 xkHz。
- DutyCyclePercent: 设置为 50%，表示 PWM 输出信号的高电平时间占周期的 50%。
- GTIOCAOutputEnabled: 设置为 True，启用 GTIOCA 引脚的输出功能。
- GTIOCAStopLevel: 设为 PinLevelLow，表示定时器停止时 GTIOCA 输出低电平。
- GTIOCBOutputEnabled: 设置为 True，启用 GTIOCB 引脚的输出功能。
- GTIOCBStopLevel: 设为 PinLevelLow，表示定时器停止时 GTIOCB 输出低电平。

Project7: AI 辅助串口配置

1.了解串口

串口通信 (UART) 是一种常用的异步数据传输方式，在嵌入式系统中广泛应用于设备之间的数据交互。本文主要探讨如何通过串口接收数据，包括初始化配置、数据接收流程、常见问题处理以及实现串口数据接收的实际代码示例。

2.对话 AI 导师

告诉 AI 导师我们已经完成了上一步的配置，接下来在 AI 导师的指引下进行串口配置：

第 6 课 / 共 17 课 · 串口 (UART) 配置

1. 和定时器一样，先创建组件：点击 **New Stack** → **Connectivity** → **UART (r_sci_uart)**。
2. 选中新加的 UART 模块，在 **Module xxx UART (r_sci_uart)** 属性里配置：

General 里：

- Name: **Blue_Tooth**
- Channel: **0**
- Data Bits: **8bits**
- Parity: **None**
- Stop Bits: **1bit**

Baud 里：

- Baud Rate: **9600**
- Baud Rate Modulation: **Enable**

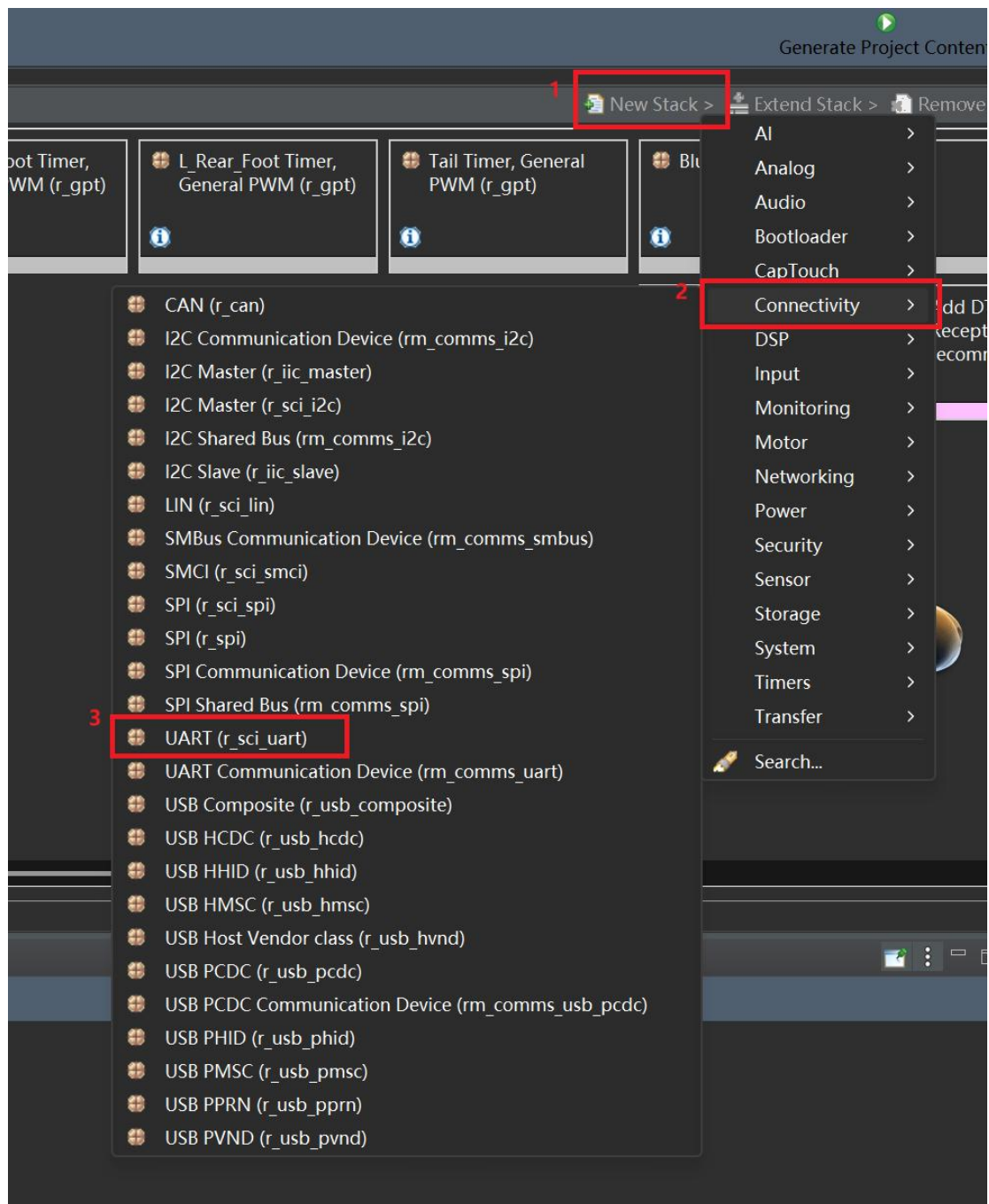
Interrupts 里：

- Callback: **Blue_Tooth_Callback**

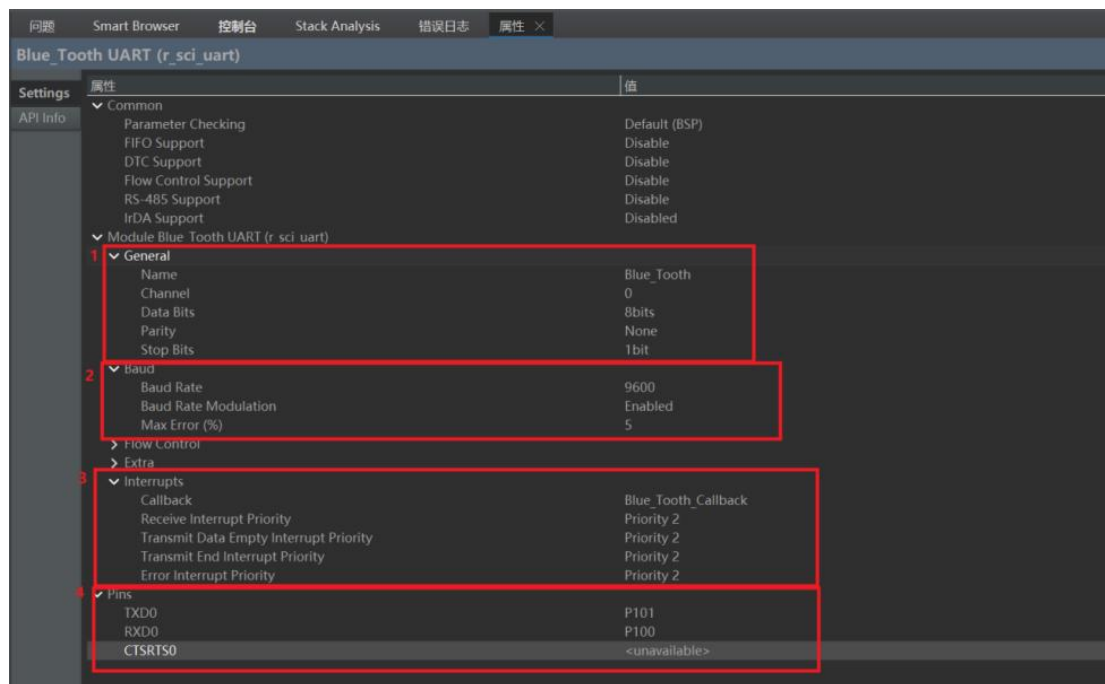
Pins 里：

- TXD0: **P101**
- RXD0: **P100**
- CTSRTS0: **<unavailable>**

3.创建组件



4.属性配置



配置引脚时，跳转到引脚配置界面后，首先将模式配置为异步模式，接着将 TX 配置为 P101，将 RX 配置为 P100

Pin Configuration				
Name	Value	Lock	Link	
Pin Group Selection	Mixed			
Operation Mode	Asynchronous UART			
Input/Output				
TXD0	✓ P101	✓	→	
RXD0	✓ P100	✓	→	
SCK0	None	✓	→	
SDA0	None	✓	→	
SCL0	None	✓	→	
CTSRTS0	None	✓	→	

5.属性介绍

Name 名称：该 UART 外设实例在工程中的唯一标识，用于代码中调用串口相关 API。

Channel 通道：选择使用 MCU 的第 90 号 SCI 串口外设（UART0）。

Data Bits 数据位：设置串口每帧数据为 8 位，是最常用的串口数据格式。

Parity 无校验位：关闭串口数据的奇偶校验功能，简化通信协议。

Stop Bits 停止位：设置每帧数据后跟随 1 个停止位，是标准串口通信配置。

Baud Rate 波特率：设置串口通信速率为 9600bps，与我们所选用的蓝牙模块的波特率相符。

Baud Rate Modulation 启用波特率调制：开启波特率微调功能，提升高速通信下的时钟误差容忍度。

Callback 回调函数名称：串口中断触发后自动调用的用户自定义函数，用于处理收发数据。

TXD0 串口发送端口：分配 P101 引脚作为 UART0 的 TXD 发送引脚，用于向外发送数据。

RXD0 串口接收端口：分配 P100 引脚作为 UART0 的 RXD 接收引脚，用于接收外部数据。

Project8:AI 辅助 I2C 配置

1.了解 I2C

本章旨在通过 I2C 接口驱动 OLED 显示屏（常见型号如 SSD1312），实现图形和文本的显示功能。OLED 屏幕具有低功耗、自发光、对比度高等优点，适用于嵌入式人机界面显示。在本项目中我们通过 I2C 驱动 OLED 屏幕来显示小马的表情与当前的状态。

2.对话 AI 导师

第 7 课 / 共 17 课 · I2C 配置与生成代码

1. 继续点击 **New Stack** → **Connectivity** → **I2C Master (r_iic_master)**。

2. 在属性里设置：

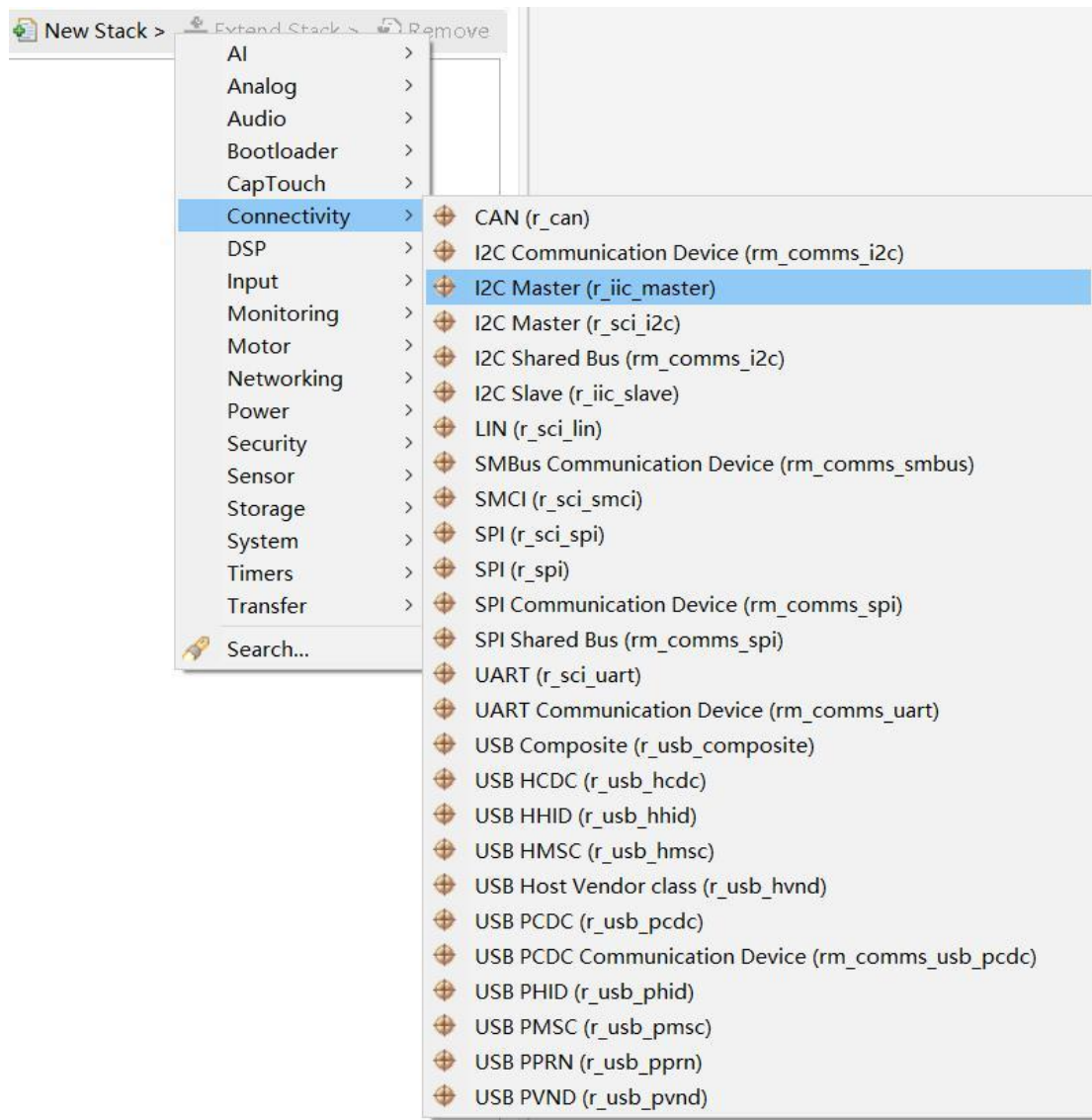
- Name: **OLED**
- Channel: **0**
- Rate: **Fast-mode**
- Custom Rate (bps): **400000**
- Slave Address: **0x3C**
- Address Mode: **7-Bit**
- Callback: **OLED_callback**
- Interrupt Priority Level: **Priority 1**

3. 配置引脚：SCL 选 **P408**，SDA 选 **P407**。

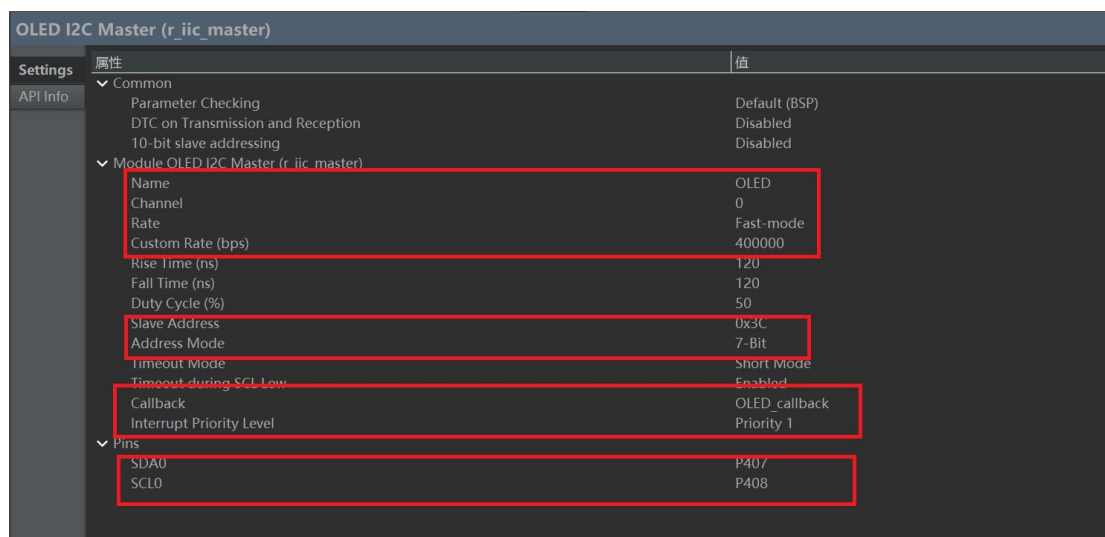
4. 最关键的一步：全部配好后，点击右上角 **Generate Project Content** 生成底层驱动代码！

生成之后工程里会出现 **ra_gen/** 目录，里面是根据你的配置自动产生的初始化代码。

3. 创建组件



4.属性配置



5.属性介绍

名称 (Name)：设置为 g_i2c_master0，表示这是 I2C 主机的实例名称，用于代码中引用。

通道 (Channel)：设为 0，表示使用的是 IIC 通道 0。

通讯速率模式 (Rate)：设置为 Fast-mode，即使用 400kHz 作为 I2C 通信速度。

自定义速率 (CustomRate(bps))：精确设置通信速率为 400000bps。

设备地址 (SlaveAddress)：设为 0x3C，表示从设备的 7 位地址，常见于 OLED 屏。

地址位数模式 (AddressMode)：设置为 7-Bit，即使用标准的 7 位地址。

超时模式 (TimeoutMode)：设置为 ShortMode，表示启用短时超时机制。

回调函数 (Callback)：设置为 i2cmastercallback，用于在通信完成或出错时触发处理函数。

中断优先级 (InterruptPriorityLevel)：设置为 Priority12，用于控制中断响应优先级。

SDA 管脚 (SDA0)：分配在 P407，引脚用于数据传输线。

SCL 管脚 (SCL0)：分配在 P408，引脚用于时钟线。

6.Generate

在完成所有的图形化配置后一定要点击右上角的 [Generate Project Content](#)，它的核心作用是将你在 FSP 配置视图中的所有图形化设置，转换为实际的项目源代码和配置文件。

Project9:AI 辅助编写 PWM 驱动

1.了解 PWM 相关函数

R_GPT_Open()

用于初始化通用 PWM 定时器（GPT）模块，并应用相应的配置。它的主要作用是启动定时器，使其按照设定的模式工作，例如周期模式（PeriodicMode）或 PWM 模式(PulseWidth ModulationMode) 。

R_GPT_Start()

用于启动 GPT（GeneralPWMTimer）定时器，使其开始按照之前的配置运行。

必须先调用 R_GPT_Open(), 否则 R_GPT_Start()会返回 FSP_ERRNOT_OPEN 错误。

在 PWM 模式下，启动定时器后会按照设定的周期和占空比输出 PWM 波形。

R_GPT_PeriodSet()

R_GPT_PeriodSet()该函数用于设置定时器的周期值。若定时器正在运行，则在下次溢出时更新周期；如果定时器已停止，该函数将重置计数器并更新周期。

R_GPT_DutyCycleSet()

RGPTDutyCycleSet 此函数用于设置请求引脚的占空比。占空比在缓冲寄存器中更新，更新后的占空比将在下一个计数溢出时反映出来。

R_GPT_Reset()

R_GPT_Reset()此函数用于将计数器值重置为 0。它实现了 timerapi_t::reset 操作。

注意：如果在上次调用 RGPTPeriodSet()后未发生计数器溢出，函数将更新为新的周期值。

R_GPT_Close()

RGPTClose 此函数用于停止计数器、禁用输出引脚并清除内部驱动程序数据。它实现了 timerapi_t::close 操作。

2.对话 AI 导师

告诉 AI 导师我们已经完成上一步配置，此时 AI 导师将给出编写完成的代码，我们按照教学对话框中的指引进行操作：

第 8 课 / 共 17 课 · 编写 PWM 驱动

在左侧文件树里选中 `src` 文件夹，右键 → 新建文件，分别建出 `PWM.c` 和 `PWM.h`。（新建 `.h` 时本软件会自动帮你写好头文件保护。）

这两个文件是整个项目的底层：把“我要某条腿转到某个角度”翻译成对定时器占空比的操作。

先想清楚三件事，再动手写：

- 定时器要怎么启动？
- 角度和占空比之间是什么关系？
- 每条腿应该暴露一个什么样的函数给上层调用？

写的过程中随时问我。

第 9 课 / 共 17 课 · 编写动作逻辑

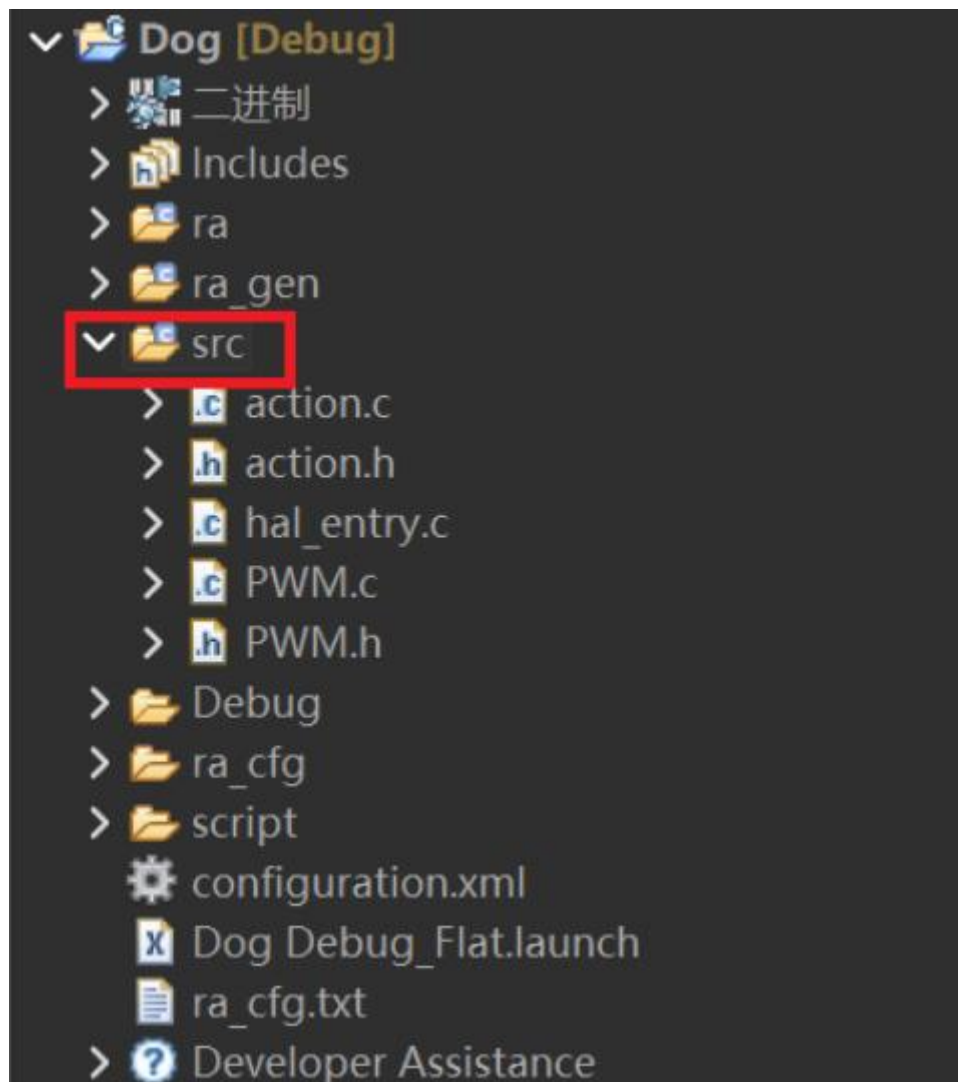
在左侧文件树里选中 `src` 文件夹，右键 → 新建文件，分别建出 `action.c` 和 `action.h`。

这一层负责把底层的 PWM 组合成“站立”“趴下”“前进”“转弯”“摇尾巴”这些动作。

建议从最简单的开始：先让机器马能站立，再做趴下，最后才是需要多条腿协同、按时序交替的前进。

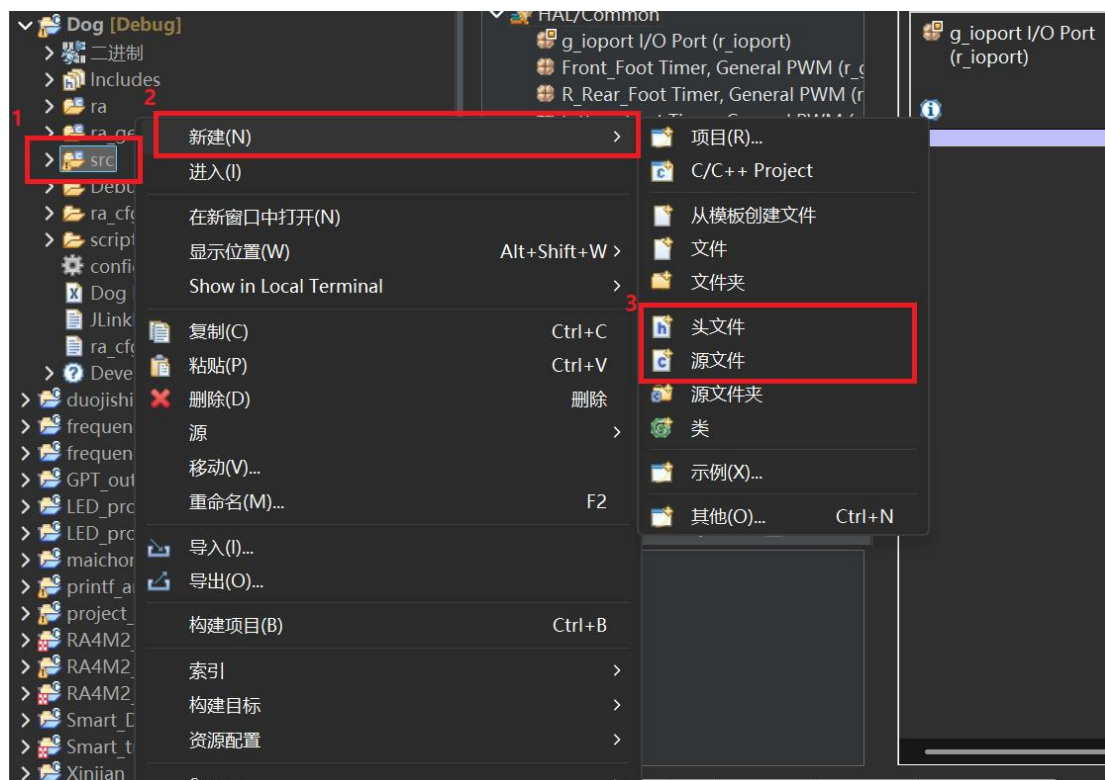
写的过程中随时问我。

3.src 文件位置



4.创建源文件（.c）与头文件(.h)

(1) 右键 src 文件夹创建源文件（.c）与头文件(.h)



(2) 创建头文件



(3) 创建源文件



5.代码移植

将 AI 导师左侧代码生成区的代码复制到源文件与头文件中

Project10：AI 辅助编写动作逻辑

1.了解舵机工作机制

舵机使用 50Hz（周期 20ms）的 PWM 信号控制，占空比 2.5%~12.5%线性对应 0°~180°。其中 2.5%对应高电平 0.5ms（0°），12.5%对应高电平 2.5ms（180°）。当输出 12.5%占空比时，舵机内部电路识别出 2.5ms 脉宽，驱动电机使输出轴旋转并锁定在 180°极限位置

2.对话 AI 导师

按照 AI 导师的指引创建对应的源文件与头文件，并将 AI 导师生成的代码移植到自己的工程中。

第 10 课 / 共 17 课 · 导入 OLED 库

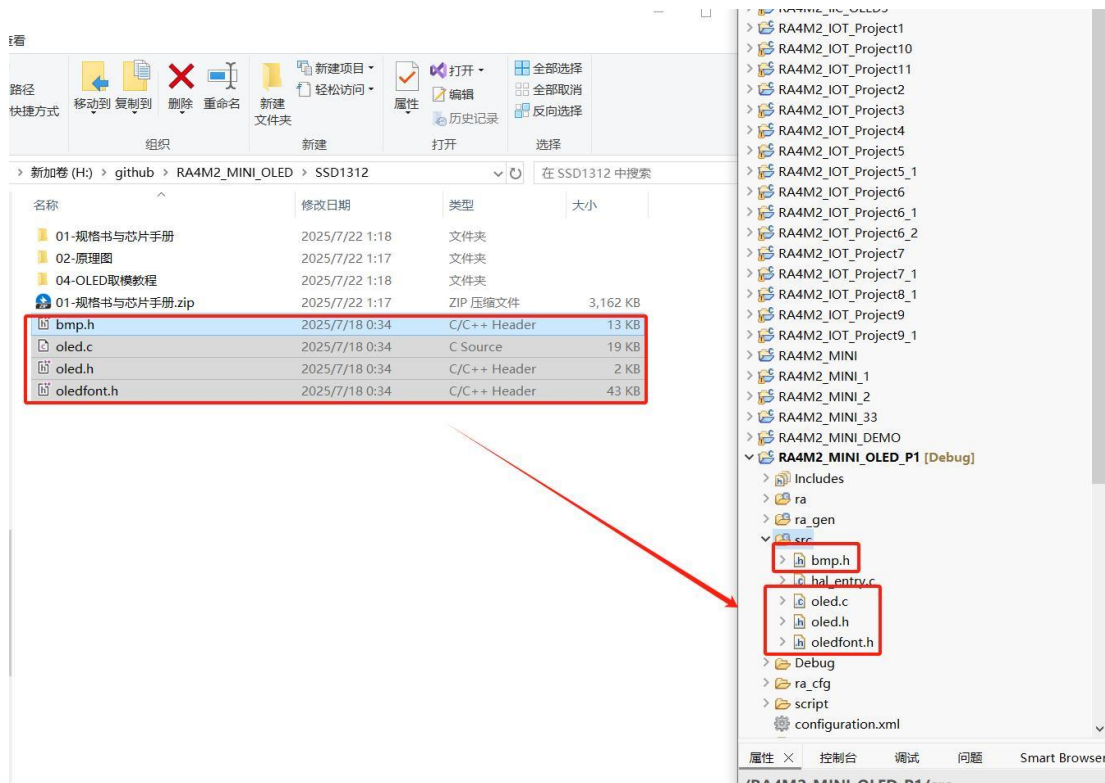
OLED 的底层驱动比较复杂，我们通常直接从外部导入现成的库文件。

找到资料包里提供的 `oled.c`、`oled.h`、`oledfont.h`，复制到工程的 `src` 文件夹中（用系统的文件管理器拷进去即可，左侧文件树会自动刷新）。

Project11：AI 辅助驱动 OLED

1. 获取驱动

复制资料包中的这四个文件，找到对应工程的文件夹，将其粘贴到 `src` 文件夹下



2. 驱动文件简介

bmp.h

这是图像数据头文件，包含多个 BMP[] 数组，例如 BMP1[]、BMP2[]、BMP3[] 等。

作用：保存点阵图像的原始像素数据，通常从 .bmp 图转换而来。

用途：用于在 OLED 上显示图标、LOGO 或图形界面。

配合函数：使用 OLEDShowPicture() 将这些图片数据渲染到屏幕上。

oled.c

这是 OLED 控制逻辑的实现文件，负责初始化和图形绘制等操作。

OLED_Init(): 初始化 OLED 显示屏。

OLED_WR_Byte()、Send_Byte(): 通过 I2C 向 OLED 发送命令或数据。

OLED_Refresh(): 将显示缓冲区 OLED_GRAM 中的数据刷新到屏幕。

OLED_DrawPoint()、OLED_DrawLine()、OLED_ShowString() 等：提供点、线、字符、字符串的绘制功能。

OLED_ShowPicture(): 显示 bmp.h 中的图片数组。

oled.h

是 OLED 驱动函数的头文件，定义了所有对外接口和常量。

oledfont.h

这是字体库文件，用于显示字符和数字时的字模数据。

内容：包含 6×8、8×16 等常用 ASCII 字符的点阵定义。

用途：配合 OLEDShowChar()或 OLED_ShowString()函数使用，按指定位置和大小在 OLED 屏幕上显示文字。

3.IC2 相关函数介绍

R_IIC_MASTER_Open() 函数为执行 IIC 初始化

R_IIC_MASTER_Write() 函数是向 IIC 设备中写入数据

OLED_WR_Byte()

函数是标准的 RenesasFSPI2COLED 写入函数，它将命令或数据写入 OLED 控制器，区别在于控制字节 0x00 或 0x40，整个过程使用硬件 I2C 保证时序稳定。

R_IIC_MASTER_Read() 函数是向 IIC 设备中读取数据

4.OLED 驱动相关函数介绍

OLED_Init()

初始化 OLED 屏幕工作状态。

发送一系列命令以配置页寻址模式、电荷泵开启、对比度调节、显示使能等。

必须在任何显示函数调用之前执行。

OLED_Clear()

清空 OLED 显存缓冲区 OLEDGRAM。

设置为全 0，表示关闭所有像素。

并通过 `OLED_Refresh()` 将清空的显存数据刷新到屏幕，确保视觉上是全黑状态。

OLED_ColorTurn(0)

设置显示颜色模式。

0 为正显（背景黑，文字白）。

1 为反显（背景白，文字黑）。

实际通过向 OLED 控制器写入 0xA6（正显）或 0xA7（反显）命令实现。

OLED_DisplayTurn(0)

设置显示方向。

0 表示正常方向； 1 表示上下翻转显示。

OLED_ShowString(84,0,"RA4M2",16,1)

在指定坐标位置（x=84,y=0）显示字符串"RA4M2"。

字体大小为 16（对应 8x16 字模）。

第五个参数为显示模式：1 表示正显。

字模数据来源于 `oledfont.h`，函数内部会将对应点阵写入显存缓冲 `OLED_GRAM`。

OLED_Refresh()

此函数将 `OLED_GRAM` 中的缓冲数据逐页写入 OLED 控制器，刷新显示内容。

若不调用本函数，`Show_String()` 写入的数据不会真正显示在屏幕上。

Project12:文字与图像取模

1.概述

本章节将介绍如何在 SSD1312 OLED 屏幕上实现中文字符和图像的显示。相较于英文字母，中文由于字符数量庞大、点阵复杂，需要使用外部字库或自定义字模来支持显示。本讲内容将围绕中文字符的点阵原理、自建简易字模库、显示函数实现方式等进行详细解析。同时，本章节也将讲解如何通过点阵方式绘制图片（如 LOGO 或

ICON)，并展示如何将 BMP 图像转换为适配 OLED 的点阵数据，实现图形显示功能。通过本讲的学习，开发者将掌握 OLED 图文混排的核心技巧，为界面美化与功能展示打下基础。

2.对话 AI 导师

第 11 课 / 共 17 课 · 文字取模

1. 打开提供的 PctoLCD2002 软件，上方菜单栏 模式(C) → 字符模式(W)。
2. 打开设置，配置如下：
 - 点阵格式：阴码
 - 取模走向：逆向
 - 取模方式：行列式
 - 输出进制：十六进制数
 - 每行显示数据：点阵 16 索引 16
 - 自定义格式：C51 格式
 - 其余默认
3. 设置完就可以取自己想要的汉字编码了。输入完毕点击“生成字模”。由于我们的单片机默认横向显示字符，而本项目需要竖屏显示，需要点击左上方的旋转，把图像旋转至竖直方向。
4. 书写规则与注意事项：打开你刚才导入的 oledfont.h，拉到最底部，把生成的代码粘贴在最后一行 #endif 的上方。注意数组命名要规范（如 Hzk1），后面的数字代表单个字模占用的字节数。

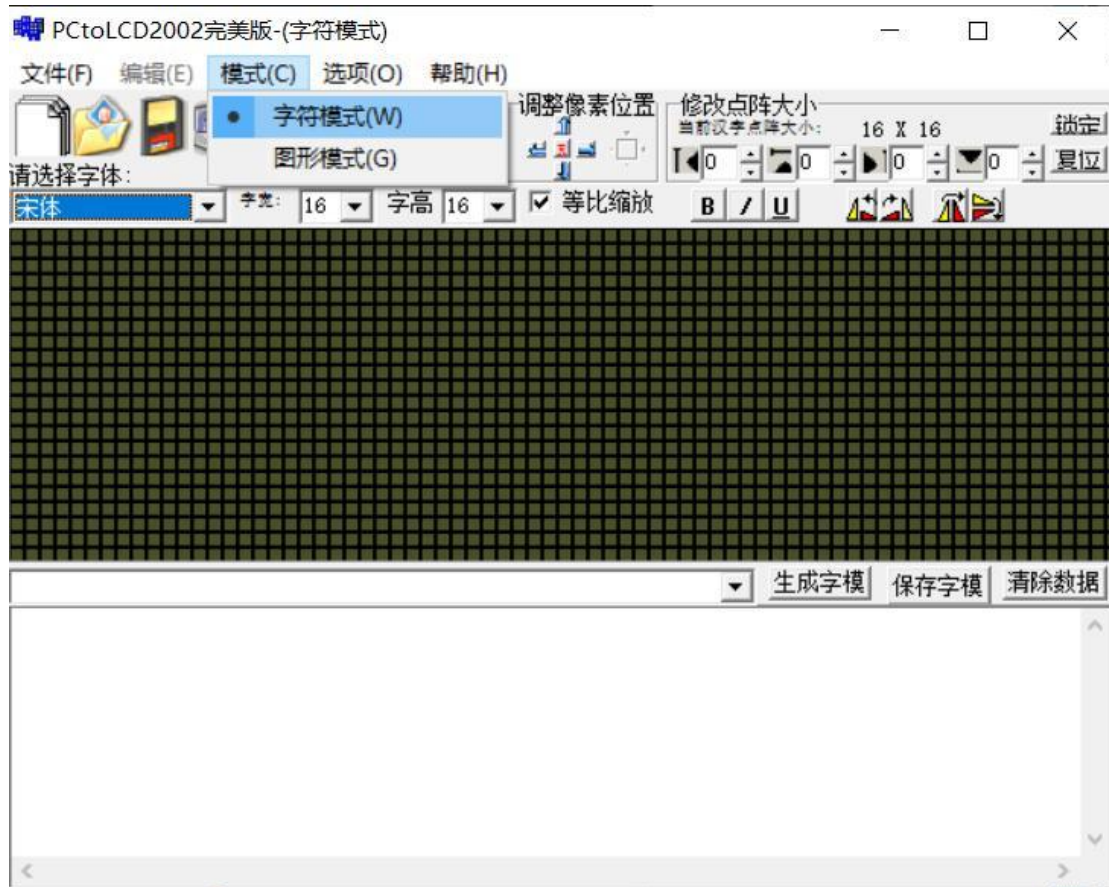
例如要在竖直方向显示“前进”两个字：先对“前”取模、旋转、保存数据，再对“进”做同样操作。

3.字符取模

- (1) 打开取模软件

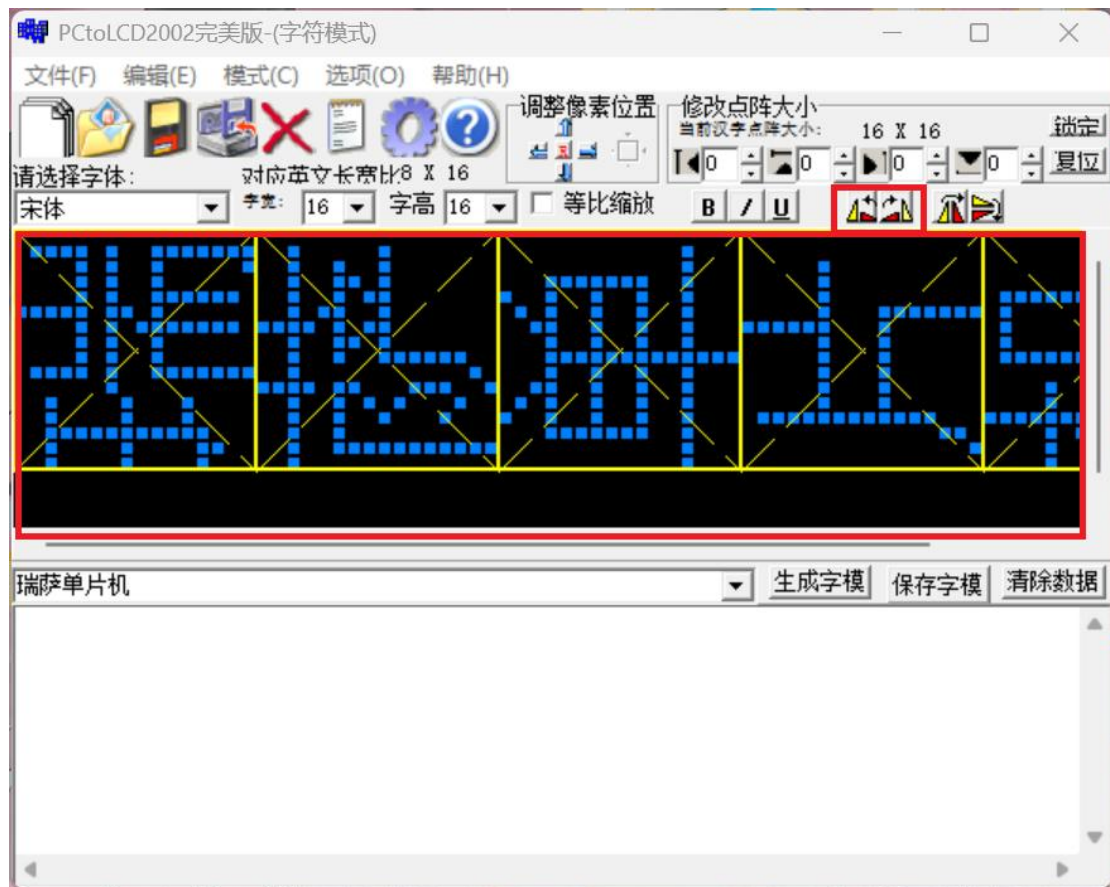


(2) 首先需要设置为字符模式



(3) 打开设置，详细配置需求

(5) 由于我们的单片机默认为横向显示字符，而本项目需要在竖屏状态下显示，我们需要点击左上方的旋转，旋转图像至竖直方向



(6) 例如我们需要在竖直方向显示“前进”两个字，我们首先对“前”取模，将其旋转，保存取模的数据，接着对“进”进行同样操作

(7) 将得到的字模存入 oledfont.h

[illegible]

(8) 其他汉字显示以此类推

4.对话 AI 导师

第 12 课 / 共 17 课 · 图像取模

1. 图像转 BMP: 取模软件只能对 bmp 格式取模。打开资料里的 `Img2Lcd.exe` , 把输出数据类型改为 **BMP** , 输出灰度调整为单色, 最大宽度和高度改为 **60x60** , 然后导入图片, 转换完成后保存。
2. 再次打开 **PCtoLCD2002** , 菜单栏 **模式(C)** → **图形模式(G)** 。
3. 打开设置: 点阵格式 **阴码**、取模走向 **逆向**、取模方式 **行列式**、输出数制 **十六进制数**、每行显示数据 **点阵 16 索引 16**、**C51 格式**。此处需要勾选 C51 格式旁边的"自定义格式", 按如下配置:
 - 段前缀: (空白)
 - 段后缀: (空白)
 - 注释前缀: `/*`
 - 注释后缀: `/*`
 - 数据前缀: `0x`
 - 数据后缀: `,`
 - 行前缀: (空白)
 - 行后缀: `,`
 - 行尾缀: `*/`
4. 打开刚转换好的 BMP 文件, 点击"生成字模"。同样, 为了竖屏显示, 需要对其进行旋转。
5. 最后在左侧文件树里选中 `src` , 右键 →  **新建文件**, 建出 `bmp.h` , 把生成的图像数据按 C 数组格式整理进去。

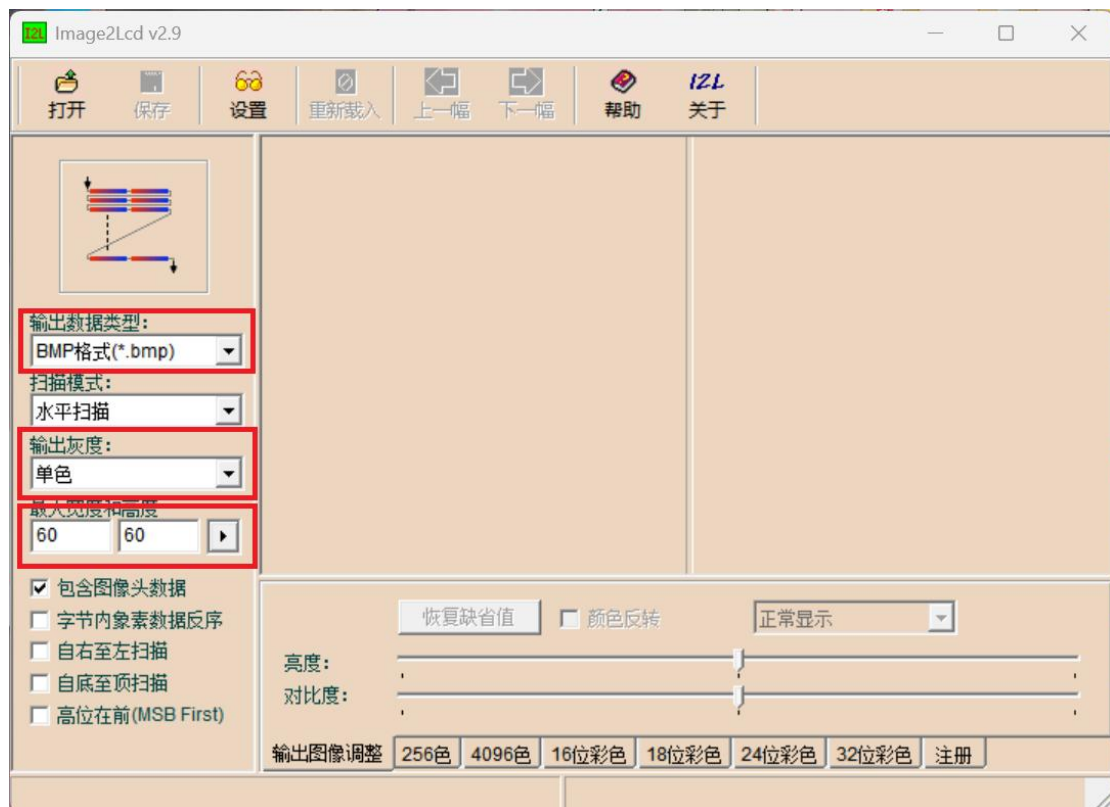
5. 图像转 BMP 格式

(1) 由于取模软件仅能对 bmp 格式的图片进行取模, 所以我们需要将资料包中给出的图片转化为 bmp 格式

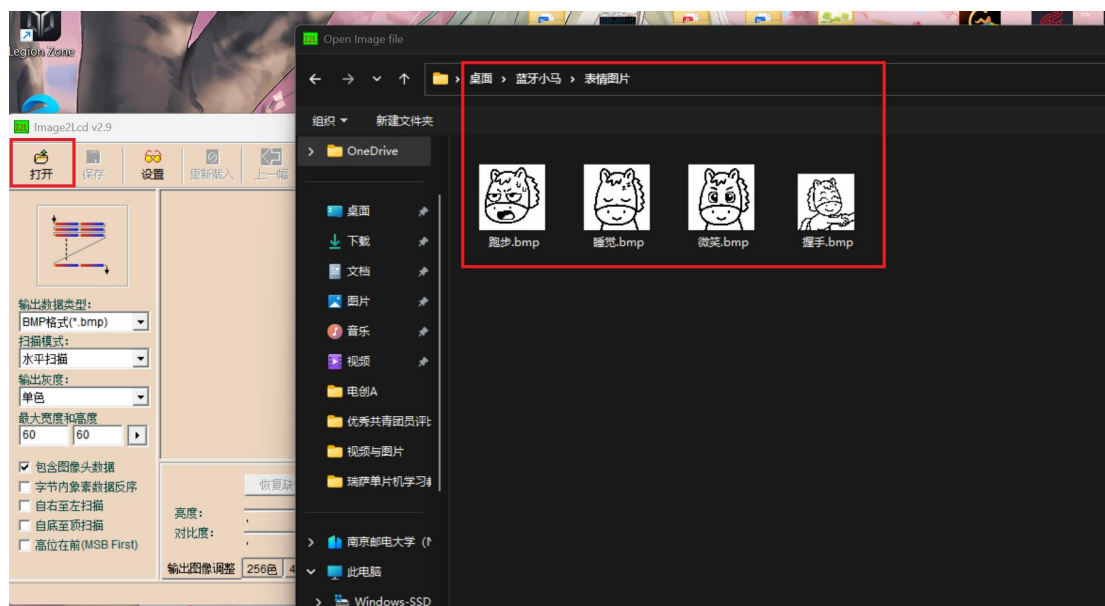
(2) 打开转格式软件



(3) 打开软件后我们将输出数据类型更改为 BMP 格式，将输出灰度调整为单色，将最大宽度和高度更改为 60x60



(4) 导入图片



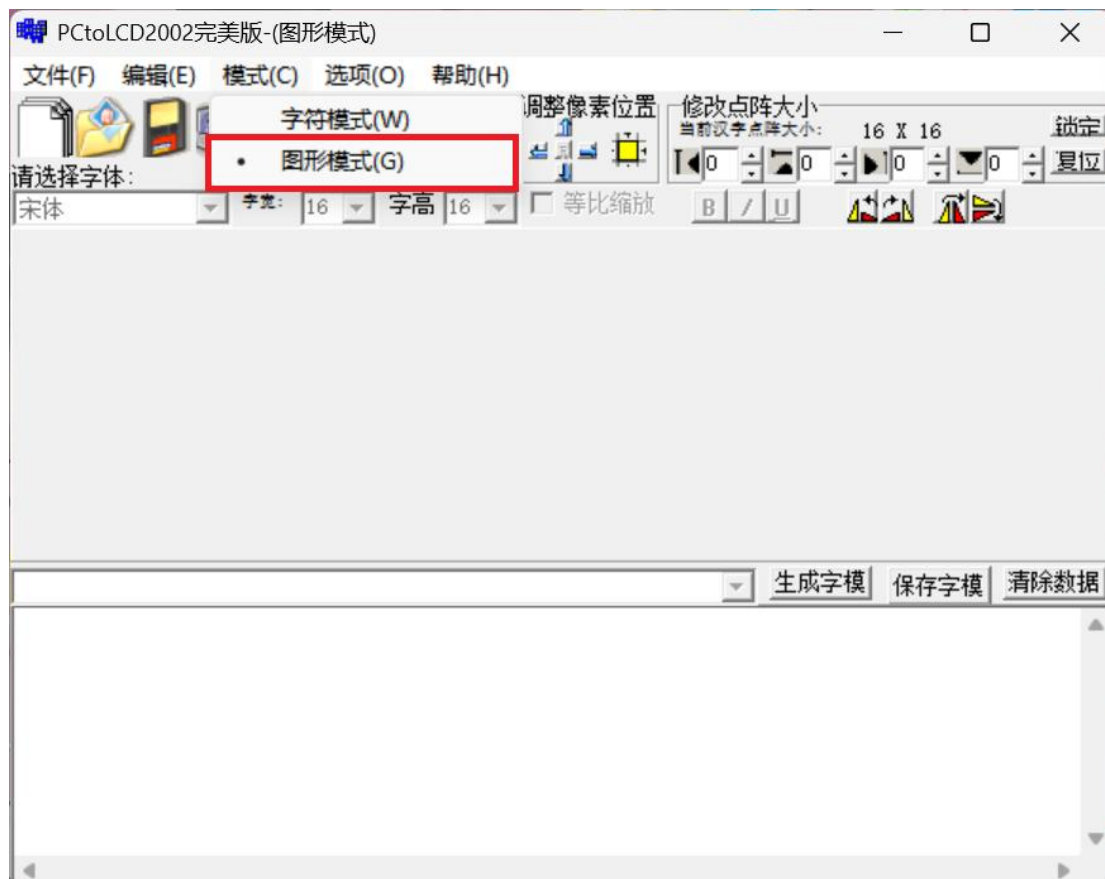
(5) 转化完成后保存图片



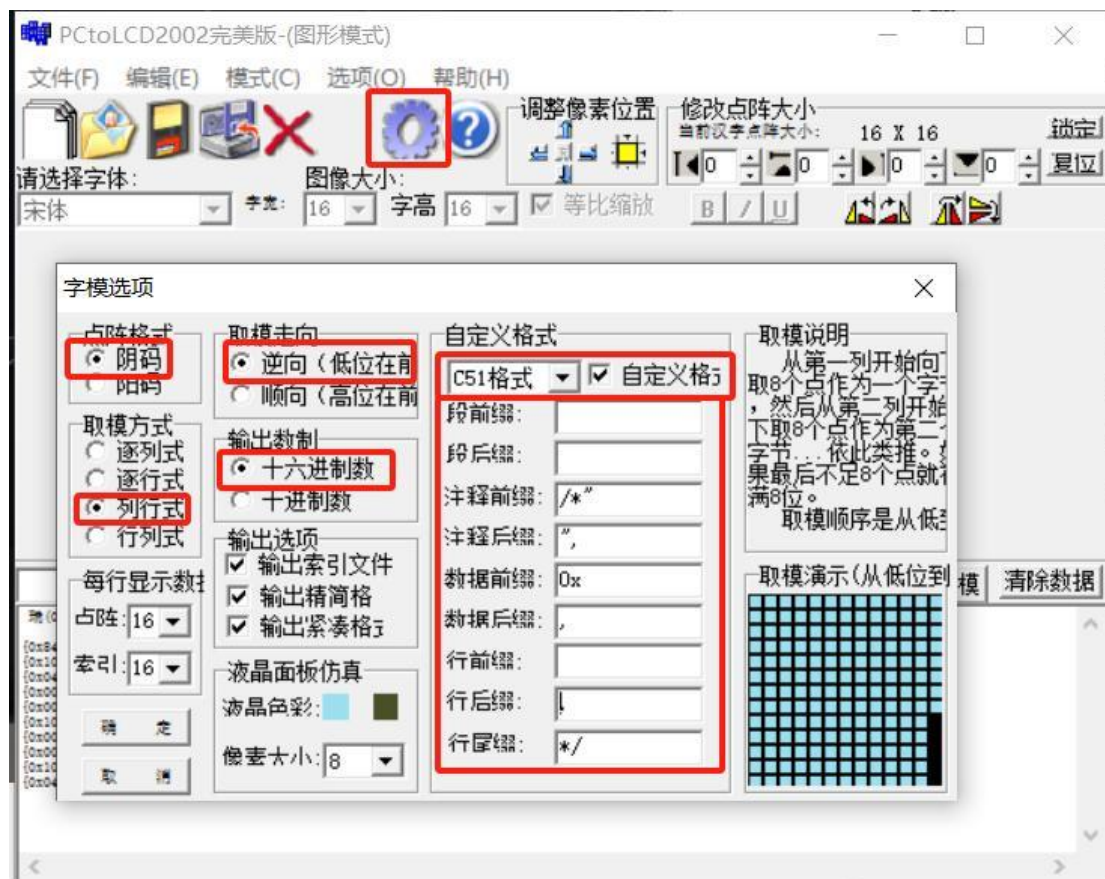
(6) 此时我们成功的将图片转化为了 BMP 格式

6. 图片取模

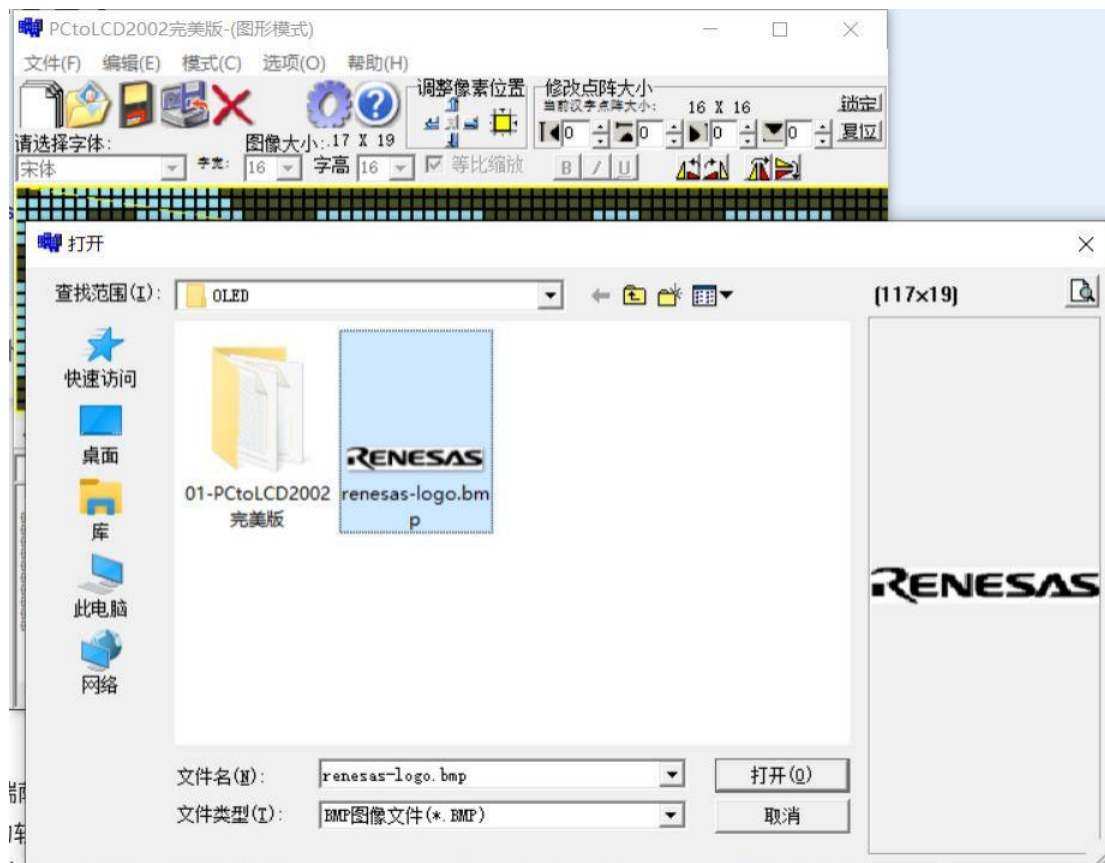
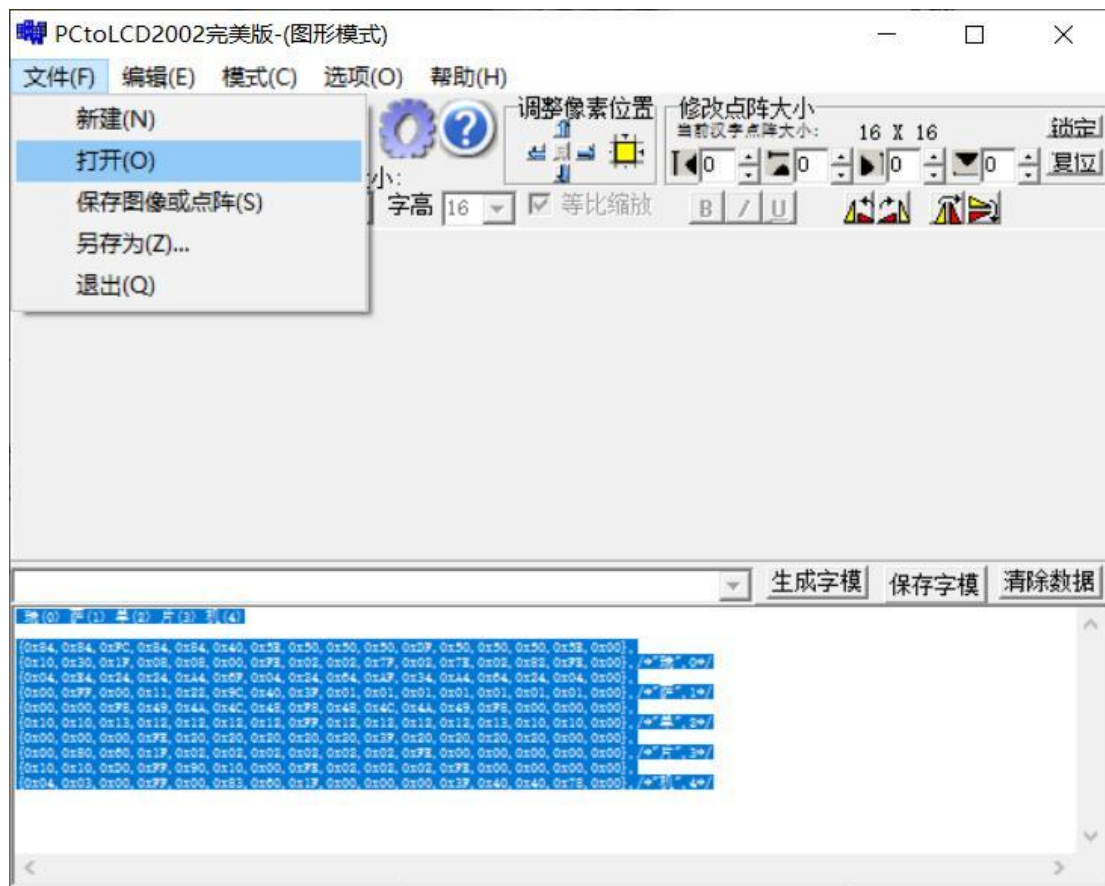
(1) 打开取模软件，设置为图形模式



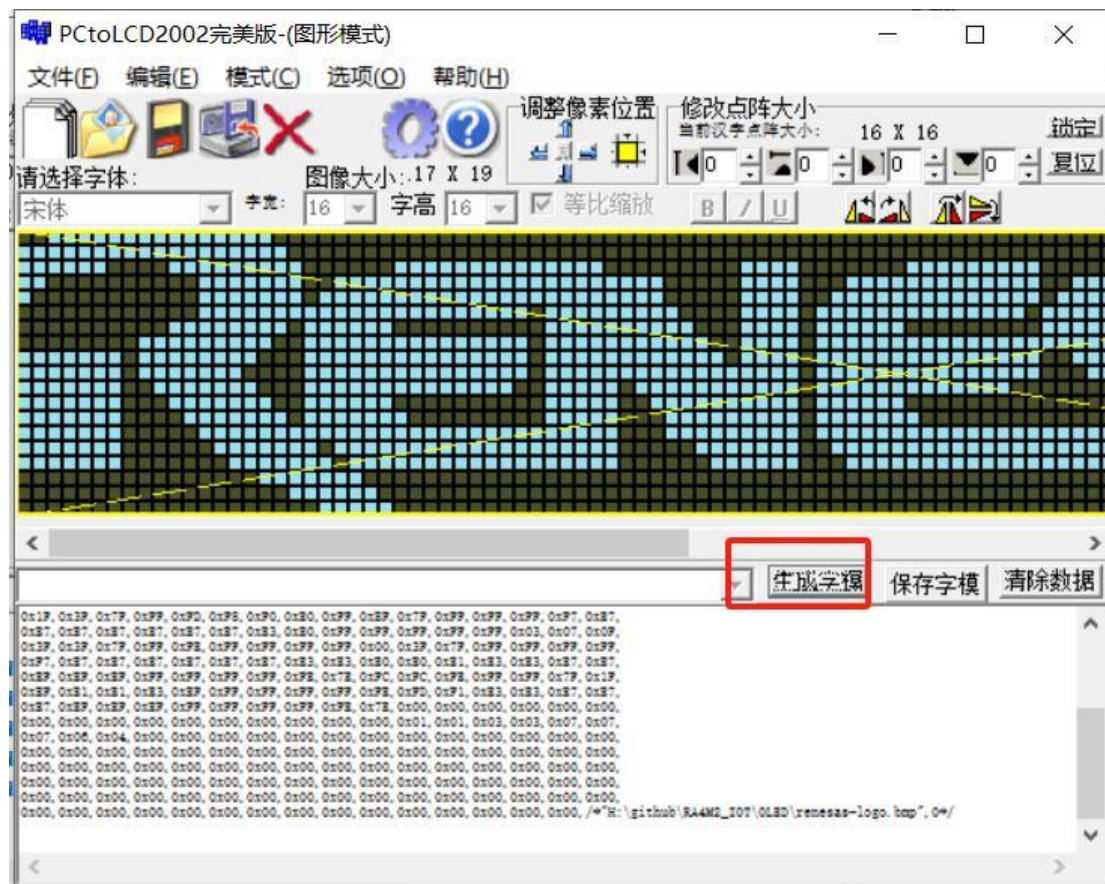
(2) 格式设置



(3) 打开刚刚转化完成的 BMP 文件

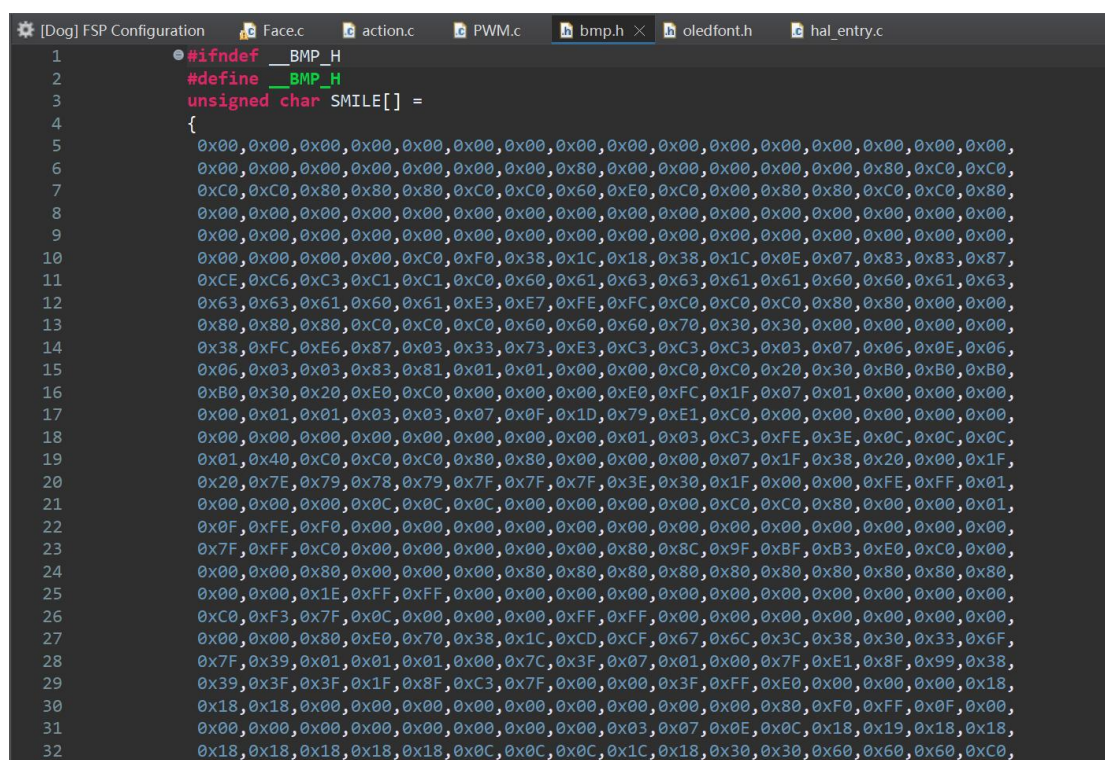


(4) 点击生成字模



(5) 同样，为了能够竖屏显示，我们也需要对其进行旋转操作

(6) 最后将生成的字模存入 bmp.h



(7) 其他图片显示以此类推

Project13: AI 辅助表情显示编写

1.对话 AI 导师

第 13 课 / 共 17 课 · 表情显示

在左侧文件树里选中 `src`，右键 → 新建文件，建出 `Face.c` 和 `Face.h`。

这一层封装 OLED 的表情显示动作：把上一步取好的字模和图像，在合适的时机画到屏幕上。

想一想：一个“表情”需要哪些参数？显示位置、图像数据、显示时长？

2.创建源文件与头文件

创建存放表情显示函数的源文件与头文件

3.代码移植

AI 导师会指引创建对应的源文件与头文件，并主动将生成的代码移植到自己的工程中。

Project14：主函数编写

1.对话 AI 导师

按照 AI 导师的指引，找到主程序并进行编写。

此时的 src 文件夹中应当存在 11 个文件，分别是主函数的源文件，动作函数的源文件与头文件，PWM 函数的源文件与头文件，OLED 驱动的源文件与头文件，字库头文件，图库头文件，表情显示的源文件与头文件

第 14 课 / 共 17 课 · 主程序编写

在左侧文件树里打开 `src/hal_entry.c`，这是 FSP 工程的用户入口。

需要在这里把前面写好的模块串起来：

- 各外设的初始化（PWM、UART、I2C/OLED）
- 蓝牙回调函数 `Blue_Tooth_Callback` 的实现——收到不同字符时执行不同动作
- 主循环里要做什么

想清楚“谁先初始化”“数据从哪里来”“动作在哪里被触发”这三个问题。

第 15 课 / 共 17 课 · 程序编译与烧录

1. 点击上位机工具栏的 🔨 一键编译 进行编译，确保日志中没有 Error。
2. 编译成功后，点击 🔥 一键烧录。

烧录前先把开发板的 BOOT 开关拨到 ON，烧录完成后拨回 OFF 并重新上电。

忘了怎么烧录随时告诉我，我再讲一遍。

2. 程序编译

在左上方菜单栏中找到一个小锤子按钮，点击，程序编译完成后若为 0 错误，则说明经过 AI 导师的指导已经成功完成了图形化配置与代码编写，此时工程已经自动生成了可烧录文件。由于驱动库本身存在的一些问题，警告提示可以忽略不计。



3. 程序烧录

参照 Project1，将编译好的函数烧录进单片机中

Project15：蓝牙串口小程序的使用

本项目采用 b 站江协科技大佬制作的蓝牙串口小程序进行操作，具体如何使用请观看资料包里附带的蓝牙串口教学视频。

第 16 课 / 共 17 课 · 蓝牙串口小程序的使用

1. 打开微信，搜索“蓝牙串口”，找到“蓝牙串口 江协科技”并打开，使用其按键模式（初始界面为 123456789）。
2. 点击右侧“编辑”，修改 7 个按键的名称和对应发送字符（均为文本形式）：
 - 前进 → 发送 2
 - 左转 → 发送 3
 - 右转 → 发送 4
 - 站立 → 发送 0
 - 趴下 → 发送 1
 - 握手 → 发送 6
 - 摇尾巴 → 发送 5

点击其他地方即可保存。

3. 关闭编辑模式，给单片机上电后，点击小程序中的“连接”，选择“开始扫描”。
4. 扫描完成后，选择名称为 BT24 的蓝牙进行配对连接。
5. 连接成功后点击对应按键，即可通过发送字符控制机器马的动作。

试一下，你的机器马听你指挥了吗？